

# Chapter 5

## Link Layer

### A note on the

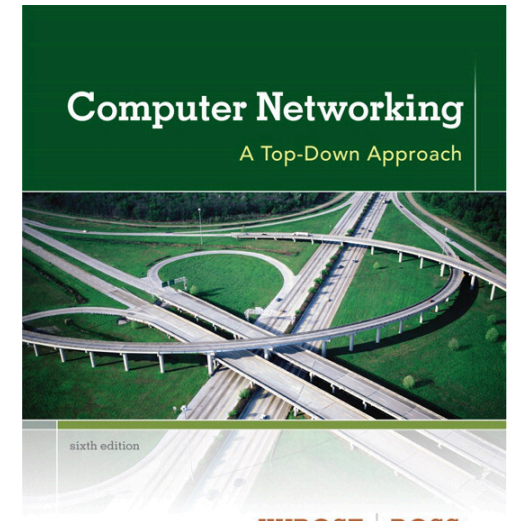
We're making these

They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a lot of work on our part. In return for use, we only ask the following:

- ❖ If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- ❖ If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2012  
J.F Kurose and K.W. Ross, All Rights Reserved



*Networking: A  
Top Down  
Approach  
6<sup>th</sup> edition  
Jim Kurose, Keith  
Ross  
Addison-Wesley  
March 2012*

# Chapter 5: Link layer

## *our goals:*

- ❖ understand principles behind link layer services:
  - acc
    - link layer addressing
    - local area networks: Ethernet, VLANs
- ❖ instantiation, implementation of various link layer technologies

# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot  
5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

life  
of a web request

# Link layer: introduction

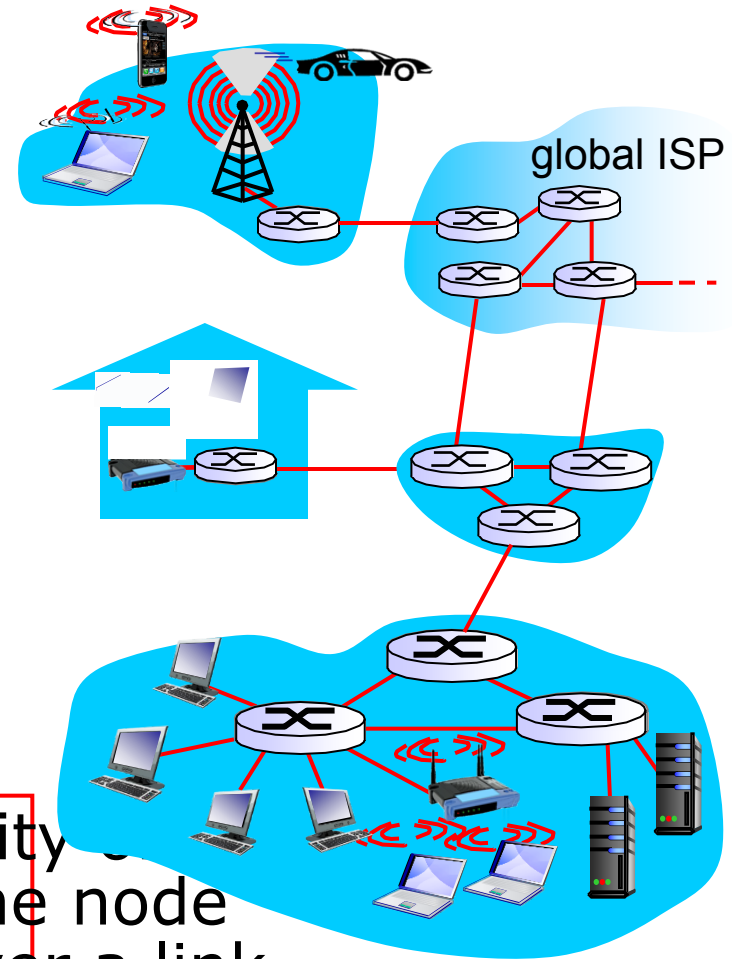
## *terminology:*

- ❖ hosts and routers:  
**nodes**
- ❖ communication

## comm links

- wired links
- wireless links
- LANs

❖ **layer-2 packet frame** has responsibility of  
encapsulating datagram from one node  
to **physically adjacent** node over a link



# Link layer: context

- ❖ datagram transferred by different link protocols over different links:

relay  
intermediate links,  
802.11 on last link

- ❖ each link protocol provides different services
  - e.g., may or may not provide rdt over link

## *transportation analogy:*

- ❖ trip from Princeton to nne

- Lausanne to
- ❖ tourist = **datagram**
  - ❖ transport segment = **communication link**
  - ❖ transportation mode = **link layer protocol**
  - ❖ travel agent = **routing algorithm**

# Link layer services

## ❖ *framing, link access:*

- encapsulate datagram into frame, adding header, trailer
- channel access

- d

## ❖ *reliable delivery between adjacent nodes*

- we learned how to do this already (chapter 3)!
- seldom used on low bit-error link (fiber, some twisted pair)
- wireless links: high error rates
  - *Q:* why both link-level and end-end reliability?

# Link layer services (more)

## ❖ *flow control:*

- pacing between adjacent sending and receiving nodes



- rece
  - signals sender for retransmission or drops frame

## ❖ *error correction:*

- receiver identifies *and corrects* bit error(s) without resorting to retransmission

## ❖ *half-duplex and full-duplex*

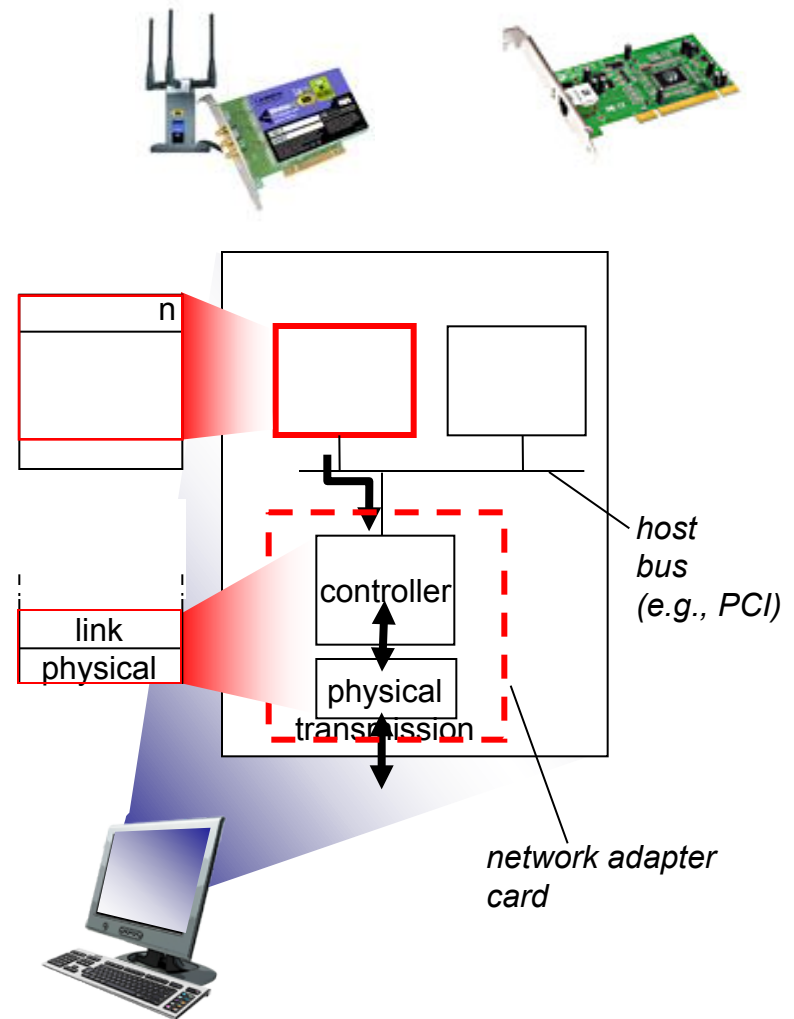
- with half duplex, nodes at both ends of link can transmit, but not at same time

# Where is the link layer implemented?

- ❖ in each and every host
- ❖ link layer implemented in “adaptor” (aka *network interface card* NIC) or on a chip

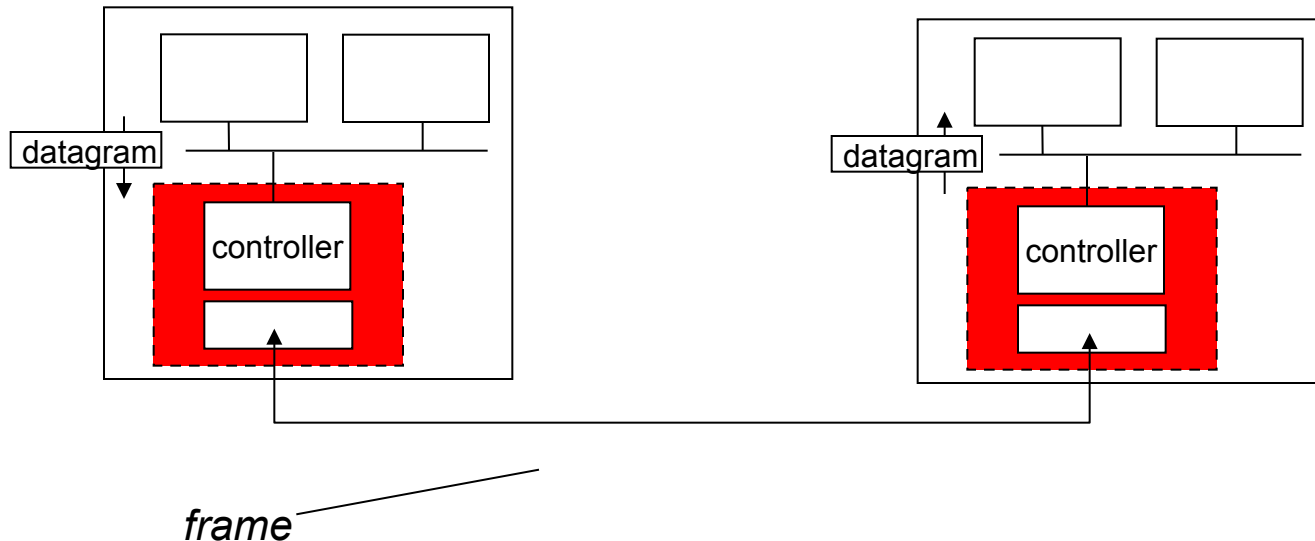
Ethernet

- implements link, physical layer
- ❖ attaches into host's system buses
- ❖ combination of hardware, software, firmware





# Adaptors communicating



## ❖ sending side:

- encapsulates datagram in frame
- adds error checking bits, rdt, flow control, etc.

## ❖ receiving side

- looks for errors, rdt, flow control, etc
- extracts datagram, passes to upper layer at receiving side

# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot  
5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

life  
of a web request

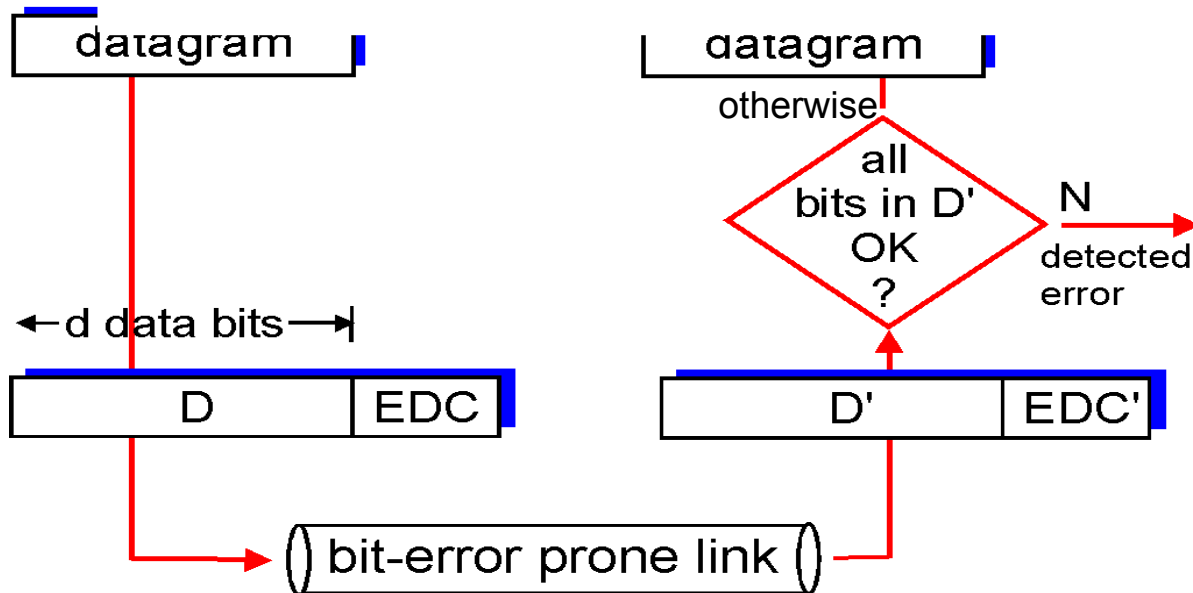
# Error detection

EDC= Error Detection and Correction bits (redundancy)

D = Data protected by error checking, may include header fields

- Error detection not 100% reliable!
  - protocol may miss

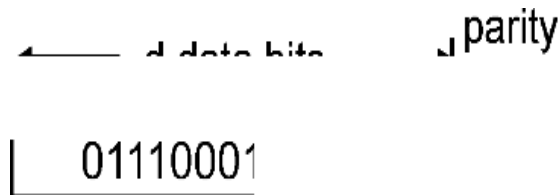
y



# Parity checking

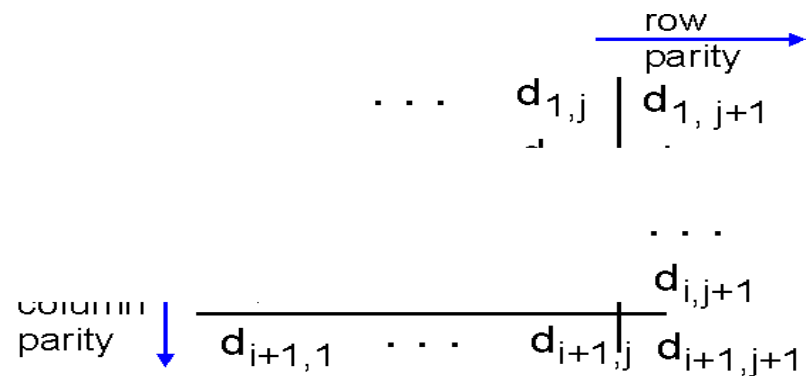
## *single bit parity:*

- ❖ detect single bit errors



## *two-dimensional bit parity:*

- ❖ detect and correct single bit errors



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 0 |

*no errors*

|              |              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|--------------|
| 1            | 0            | 1            | 0            | 1            | 1            |
| <del>1</del> | <del>1</del> | <del>1</del> | <del>1</del> | <del>0</del> | <del>0</del> |
| 0            | 1            | 1            | 1            | 0            | 1            |
| 0            | 0            | 1            | 0            | 1            | 0            |

parity error

*correctable  
single bit error*

# Internet checksum (review)

*goal:* detect “errors” (e.g., flipped bits) in transmitted packet (note: used at transport layer *only*)

*sender:*

*r:*

sequenced  
integers

- ❖ checksum: addition (1's complement sum) of segment contents
- ❖ sender puts checksum value into UDP checksum field

received  
checksum equals  
checksum field value:

- NO - error detected
- YES - no error detected. *But maybe errors nonetheless?*

# Cyclic redundancy check

- ❖ more powerful error-detection coding
- ❖ view data bits, **D**, as a binary number
- ❖ choose  $r+1$  bit pattern (generator), **G**
- ❖ goal: choose  $r$  that

rem

- can detect all burst errors less than  $r+1$  bits
- ❖ widely used in practice (Ethernet, 802.11 WiFi, ATM)

← d bits → ← r bits →



$$D * 2^r \text{ XOR } R$$

*mathematical formula*

# CRC example

want:

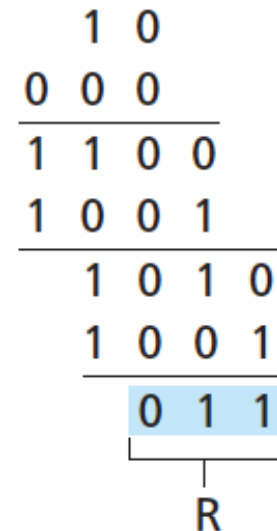
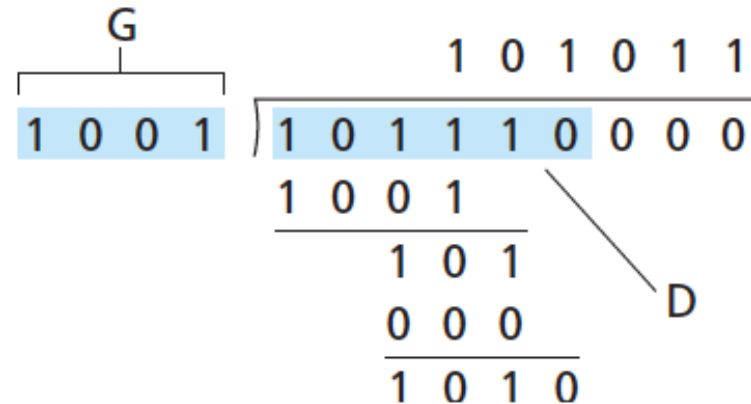
$$D \cdot 2^r \text{ XOR } R = nG$$

*equivalently:*

$\cdot 2^r$

if w  
by G, want  
remainder R to  
satisfy:

$$R = \text{remainder}\left[\frac{D \cdot 2^r}{G}\right]$$



# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot

5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

life  
of a web request



# Multiple access links, ~~protocols~~

two types of “links”:

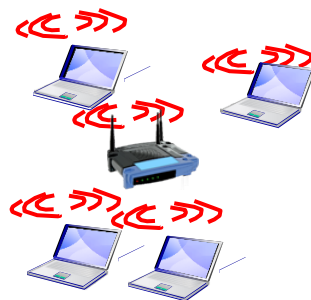
## ❖ point-to-point

- PPP for dial-up access
- point-to-point link between Ethernet switch, host

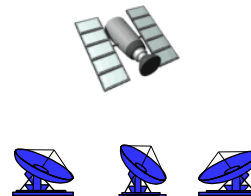
- ups
- 802.11 wireless LAN



shared wire (e.g.,  
cabled Ethernet)



shared RF  
(e.g., 802.11 WiFi)



shared RF  
(satellite)



humans at a  
cocktail party  
(shared air, acoustical)

# Multiple access protocols

- ❖ single shared broadcast channel
- ❖ two or more simultaneous transmissions by nodes: interference
  - *collision* if no or more signals at

## *multipl*

- ❖ distributed algorithm that determines how nodes share channel, i.e., determine when node can transmit
- ❖ communication about channel sharing must use channel itself!
  - no out-of-band channel for coordination

# An ideal multiple access protocol

*given:* broadcast channel of rate  $R$  bps

*desiderata:*

1. when one node transmits, it can send  
no other message
2. no collisions
3. fully decentralized:
  - no special node to coordinate transmissions
  - no synchronization of clocks, slots
4. simple

# MAC protocols: taxonomy

three broad classes:

- ❖ *channel partitioning*

- divide channel into smaller “pieces” (time slots, frequency, code)

- cha
- “recover” from collisions

- ❖ *“taking turns”*

- nodes take turns, but nodes with more to send can take longer turns

# Channel partitioning MAC protocols:

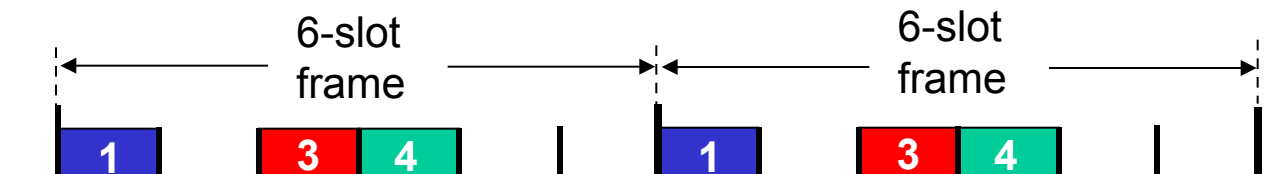
## ~~TDMA~~

### TDMA: time division multiple access

- ❖ access to channel in "rounds"
- ❖ each station gets fixed length slot  
(length = pkt) each round

❖ exam  
slots

pkt,



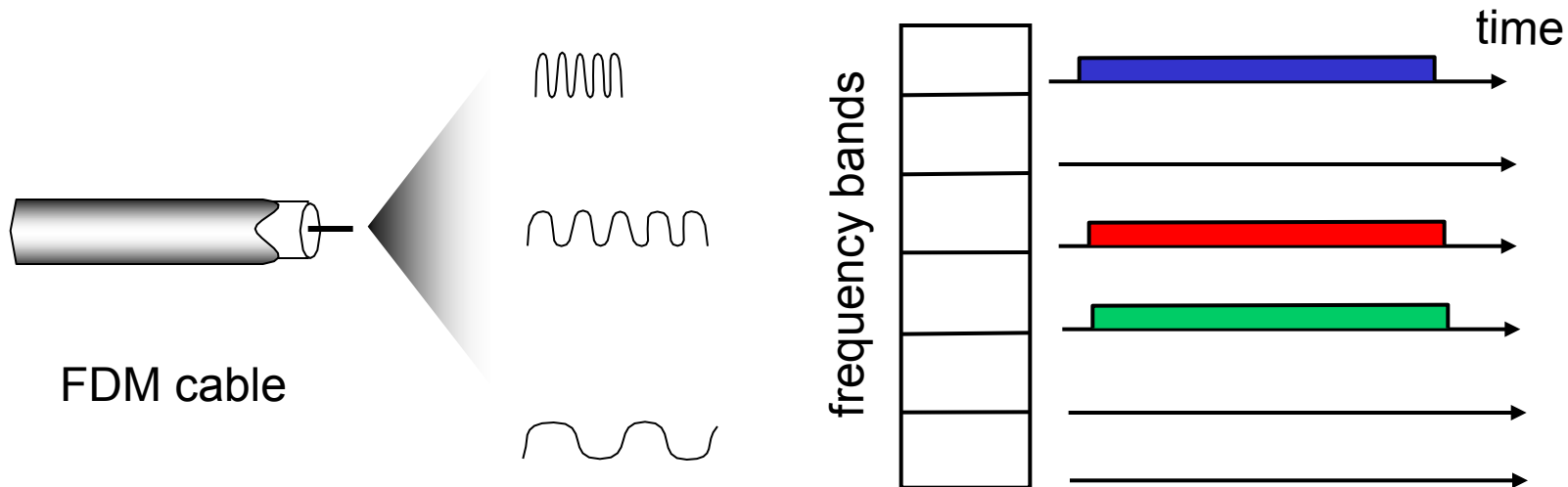
# Channel partitioning MAC protocols:

## ~~FDMA~~

### FDMA: frequency division multiple access

- ❖ channel spectrum divided into frequency bands
- ❖ each station assigned fixed frequency band
- ❖ unused transmission frequency bands go

❖ frequency



# Random access protocols

- ❖ when node has packet to send
  - transmit at full channel data rate  $R$ .
  - no *a priori* coordination among nodes

# Slotted ALOHA

## *assumptions:*

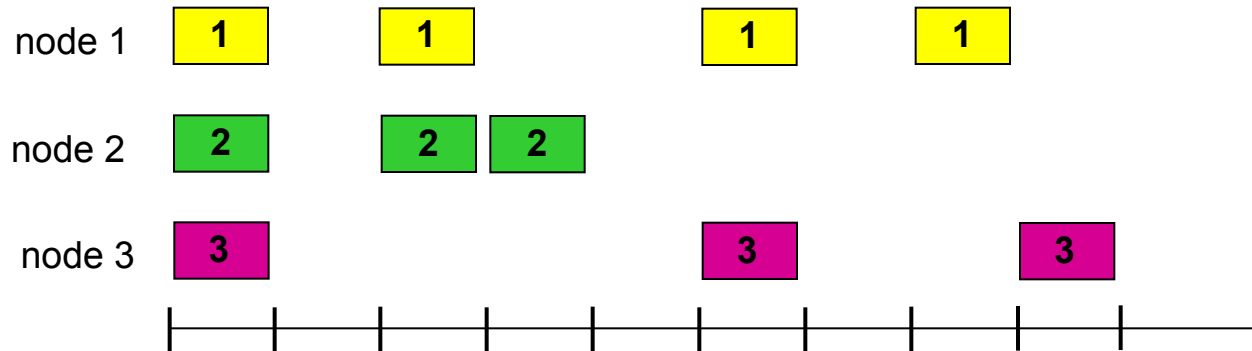
- ❖ all frames same size
- ❖ time divided into slots
- ❖ nodes transmit at the beginning of slots
- ❖ nodes are synchronized
- ❖ if 2 or more nodes transmit in slot, all nodes detect collision

## *operation:*

- ❖ when node obtains fresh frame, transmits in next slot
- *if collision:* node retransmits frame in each subsequent slot with prob.  $p$  until success



# Slotted ALOHA



## *Pros:*

- ❖ single active node can continuously transmit at full rate of channel
- ❖ highly decentralized: only slots in nodes need to be in sync
- ❖ simple
- ❖ collisions, wasting slots
- ❖ idle slots
- ❖ nodes may be able to detect collision in less than time to transmit packet
- ❖ clock synchronization

# Slotted ALOHA: efficiency

*efficiency*: long-run fraction of successful slots

- ❖ max efficiency: find  $p^*$  that maximizes  $Np(1-p)^{N-1}$

- ❖ with  $N$  nodes, each transmits in slot with probability  $p$
- ❖ prob that given node has success in a slot  $= p(1-p)^{N-1}$
- ❖ prob that *any* node has a success  $= Np(1-p)^{N-1}$

S:

$$\text{max efficiency} = 1/e = .37$$

*at best:*  
channel  
used for useful  
transmissions

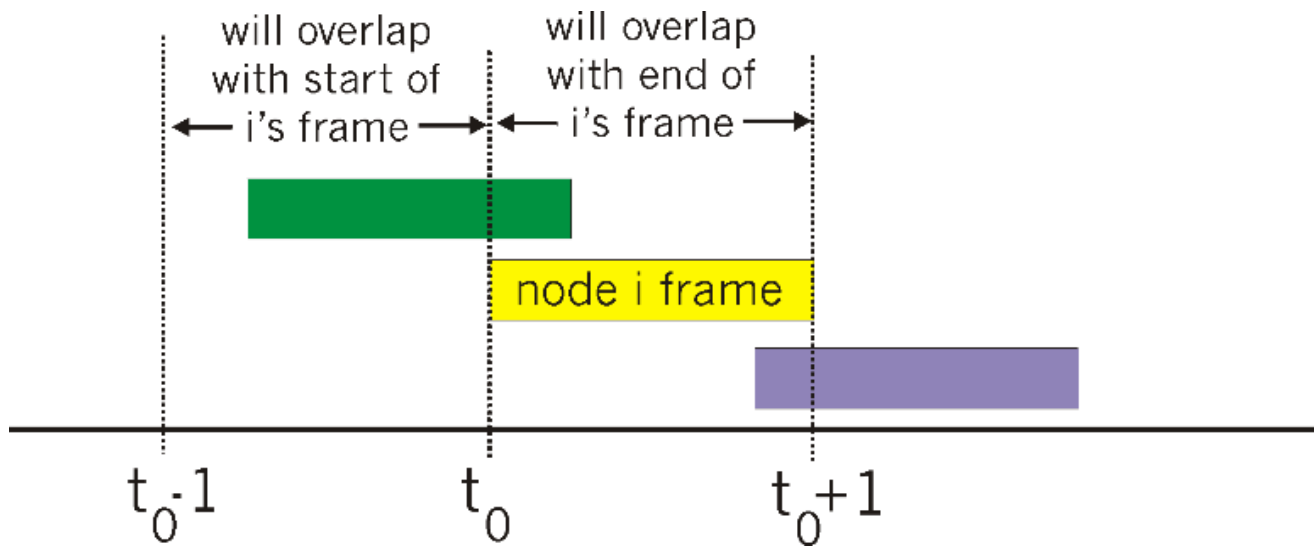
37%  
of time!



# Pure (unslotted) ALOHA

- ❖ unslotted Aloha: simpler, no synchronization
- ❖ when frame first arrives
  - transmit immediately
- ❖ collision probab

t



# Pure ALOHA efficiency

$$P(\text{success by given node}) = P(\text{node transmits}) \cdot P(\text{no other node transmits in } [t_0-1, t_0]) \cdot P(\text{no other node transmits in } [t_0, t_0+1])$$

→ ∞

... choosing optimum  $p$  and then letting  $n \rightarrow \infty$   
 $= 1/(2e) = .18$

**even worse than slotted Aloha!**

# CSMA (carrier sense multiple access)

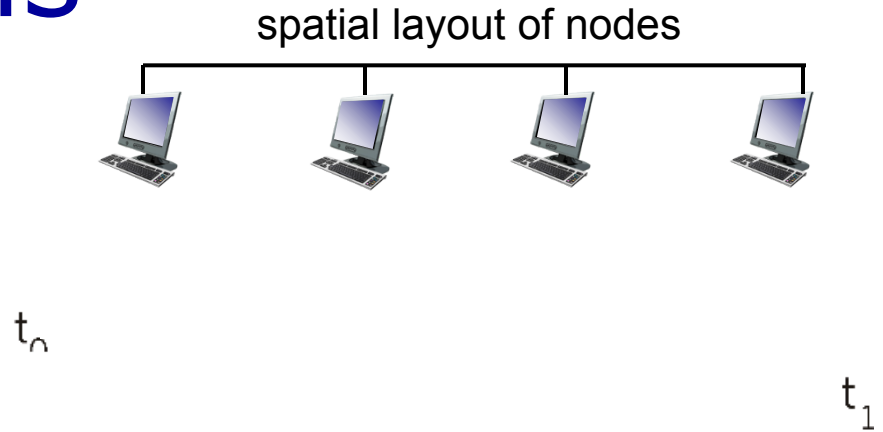
---

**CSMA:** listen before transmit:  
if channel sensed idle: transmit entire frame

- ❖ human analogy: don't interrupt others!

# CSMA collisions

- ❖ collisions *can* still occur: propagation delay means two nodes may not
- ❖ collisi  
packet transmission time wasted
  - distance & propagation delay play role in determining collision probability



# CSMA/CD (collision detection)

*CSMA/CD*: carrier sensing, deferral as in CSMA

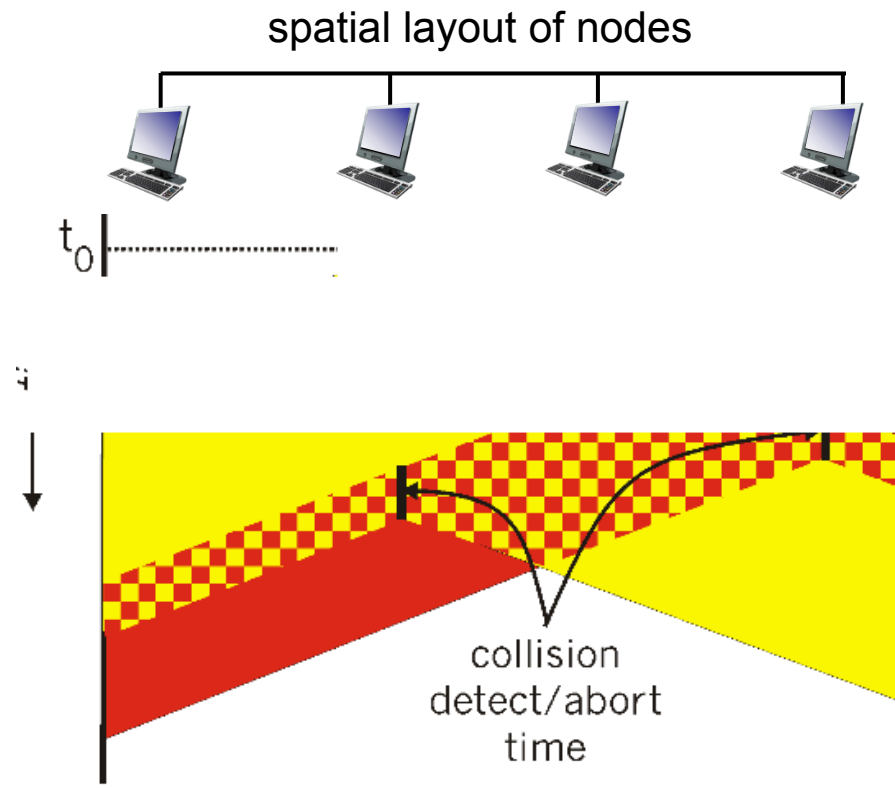
- collisions *detected* within short time

## ❖ collis

- easy in wired LANs: measure signal strengths, compare transmitted, received signals
- difficult in wireless LANs: received signal strength overwhelmed by local transmission strength

## ❖ human analogy: the polite conversationalist

# CSMA/CD (collision detection)





# Ethernet CSMA/CD algorithm

1. NIC receives datagram from network layer, creates frame
4. If NIC detects another transmission while transmitting, aborts

frame

If NIC senses channel busy, waits until channel idle, then transmits.

3. If NIC transmits entire frame without detecting another transmission, NIC is done with frame

g, NIC enters *binary (exponential) backoff*:

- after  $m$ th collision, NIC chooses  $K$  at random from  $\{0, 1, 2, \dots, 2^m - 1\}$ . NIC waits  $K \cdot 512$  bit times, returns to

# CSMA/CD efficiency

- ❖  $T_{\text{LAN}}^{\text{prop}}$  = max prop delay between 2 nodes in LAN
- ❖  $t_{\text{trans}}$  = time to transmit 1 frame

prop delay

- ❖ efficiency goes to 1
  - as  $t_{\text{prop}}$  goes to 0
  - as  $t_{\text{trans}}$  goes to infinity
- ❖ better performance than ALOHA: and simple, cheap, decentralized!

# “Taking turns” MAC protocols

## channel partitioning MAC protocols:

- share channel *efficiently* and *fairly* at high load

onl

## random access MAC protocols

- efficient at low load: single node can fully utilize channel
- high load: collision overhead

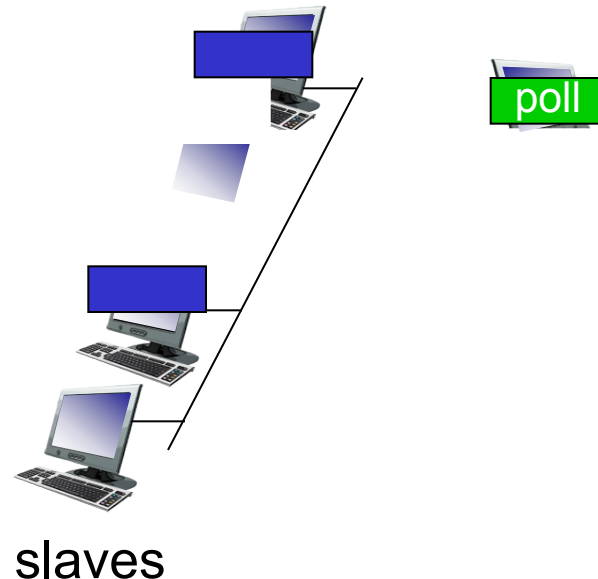
## “taking turns” protocols

look for best of both worlds!

# “Taking turns” MAC protocols

## *polling:*

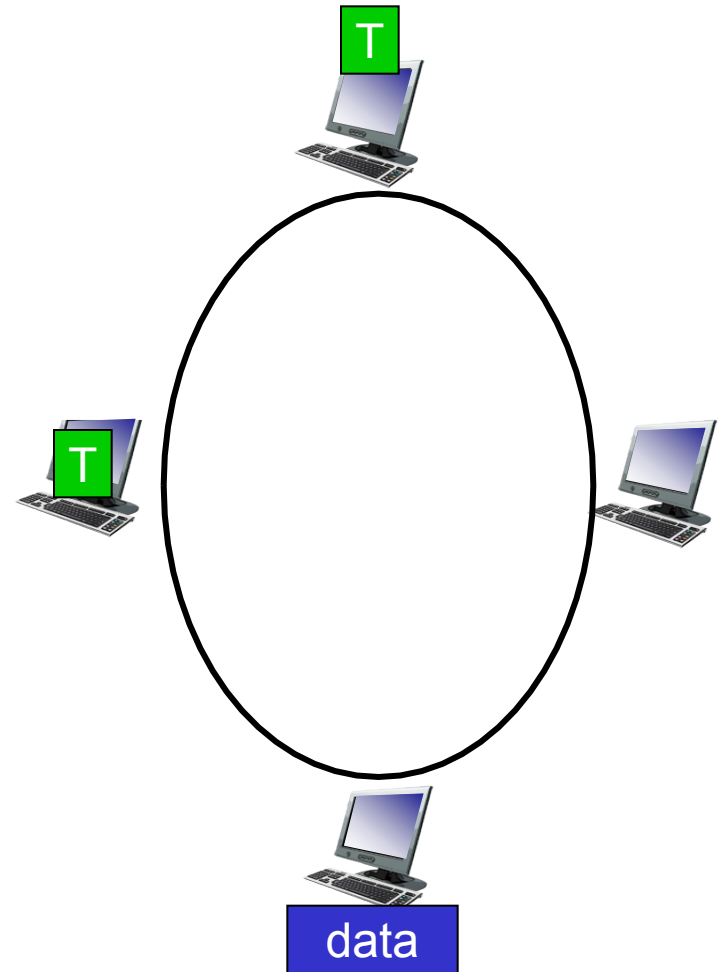
- ❖ master node “invites” slave nodes to trans
- ❖ “du devices
- ❖ concerns:
  - polling overhead
  - latency
  - single point of failure (master)



# “Taking turns” MAC protocols

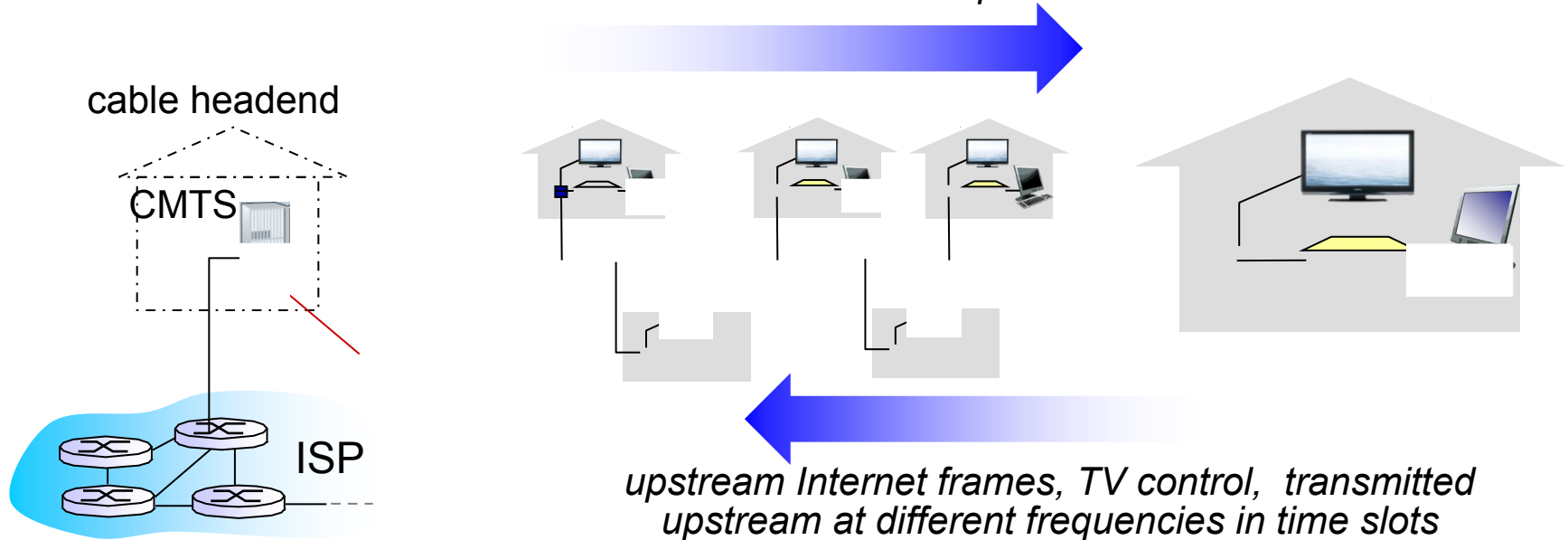
## *token passing:*

- ❖ control *token* passed from one node to next
- ❖ conce
- ❖ token overhead
- ❖ latency
- ❖ single point of failure (token)



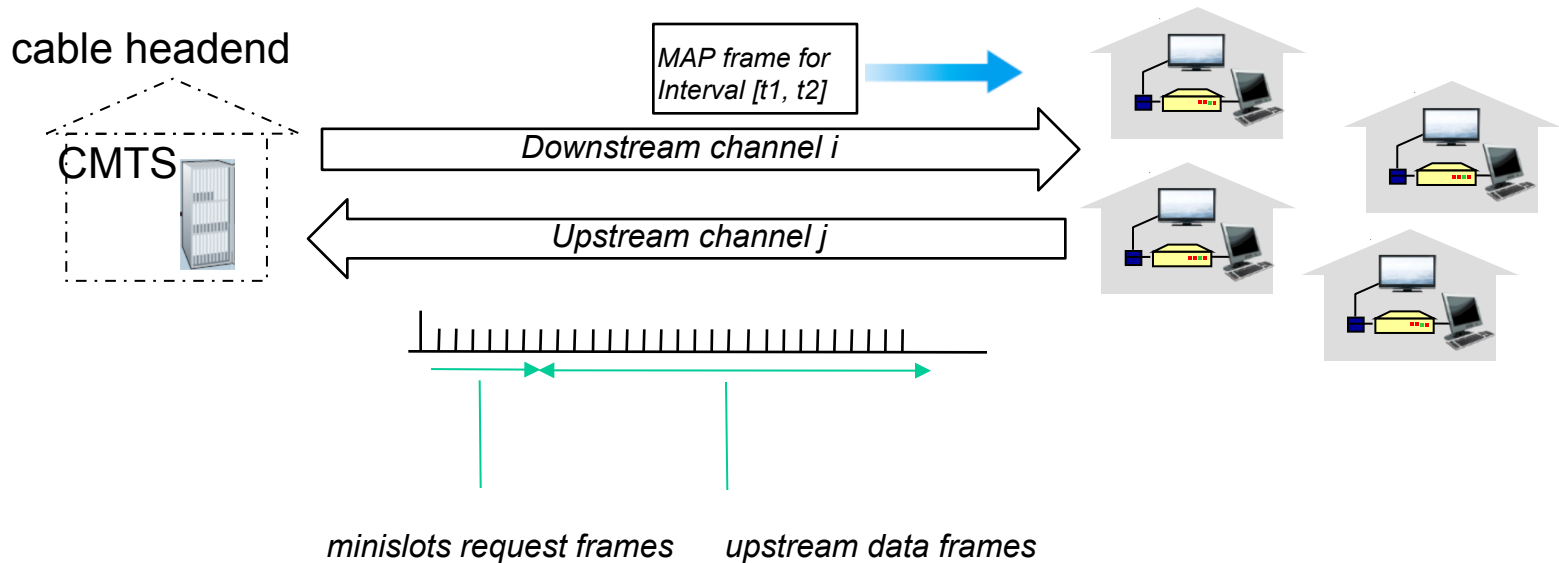
# Cable access ~~network~~

*Internet frames, TV channels, control transmitted downstream at different frequencies*



- ❖ **multiple** 40Mbps downstream (broadcast) channels
  - single CMTS transmits into channels
- ❖ **multiple** 30 Mbps upstream channels
  - **multiple access:** *all* users contend for certain upstream channel time slots (others assigned)

# Cable access ~~network~~



## DOCSIS: data over cable service interface spec

- ❖ FDM over upstream, downstream frequency channels
- ❖ TDM upstream: some slots assigned, some have contention
  - downstream MAP frame: assigns upstream slots

# Summary of MAC protocols

- ❖ *channel partitioning*, by time, frequency or code

- Time Division, Frequency Division

- ❖ *random access*

- car (wire), hard in others (wireless)

gies

- CSMA/CD used in Ethernet

- CSMA/CA used in 802.11

- ❖ *taking turns*

- polling from central site, token passing

- bluetooth, FDDI, token ring



# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot

5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

life  
of a web request

# MAC addresses and ARP

## ❖ 32-bit IP address:

- *network-layer* address for interface
- used for layer 3 (network layer) forwarding

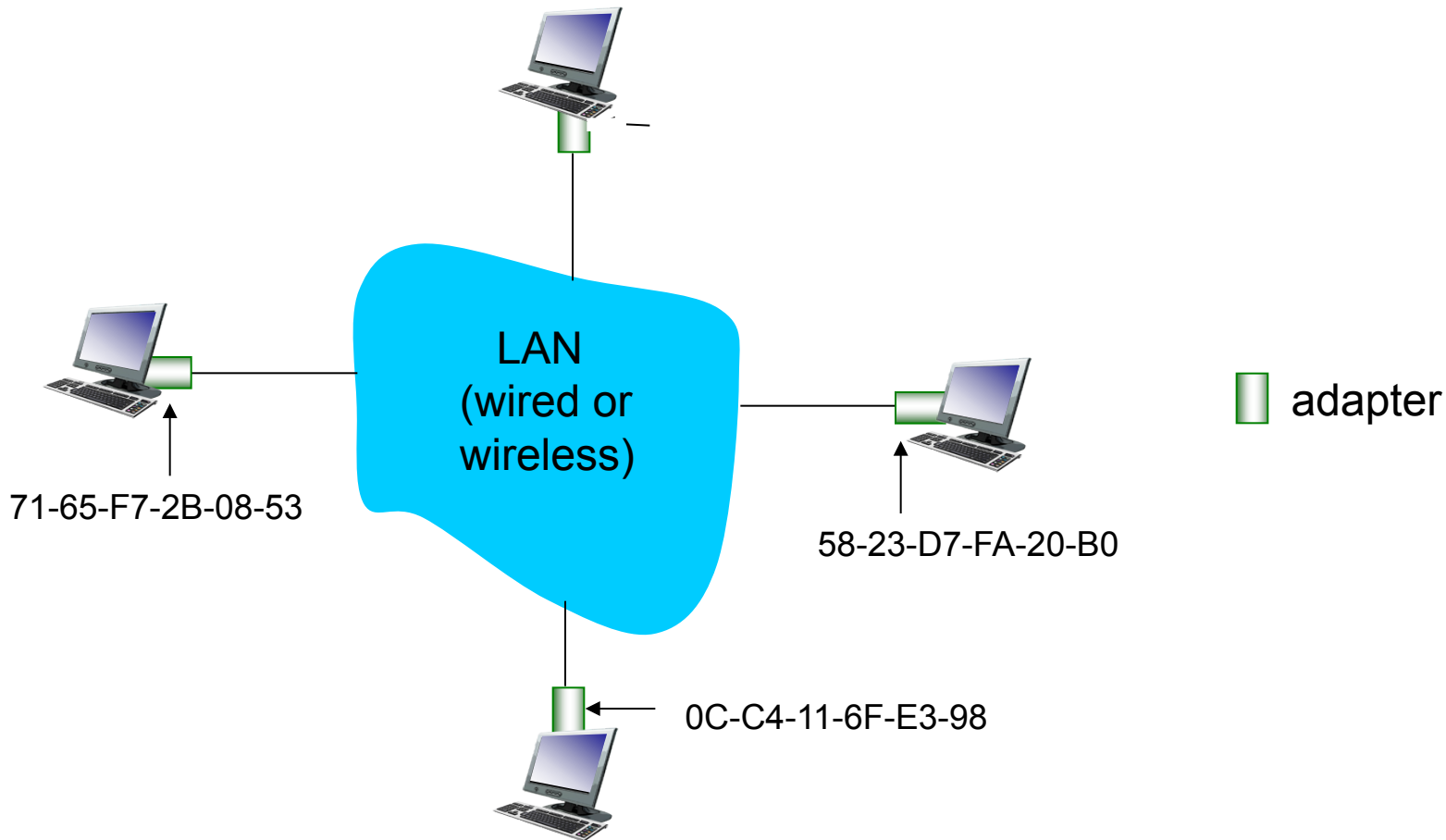
- function: *from one interface to another physically-connected interface (same network, in IP-addressing sense)*

- 48 bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable

hexadecimal (base-16) notation  
e.g.: 1A-2F-BB-76-09-AD  
(each "number" represents 4 bits)

# LAN addresses and ARP

each adapter on LAN has unique *LAN* address



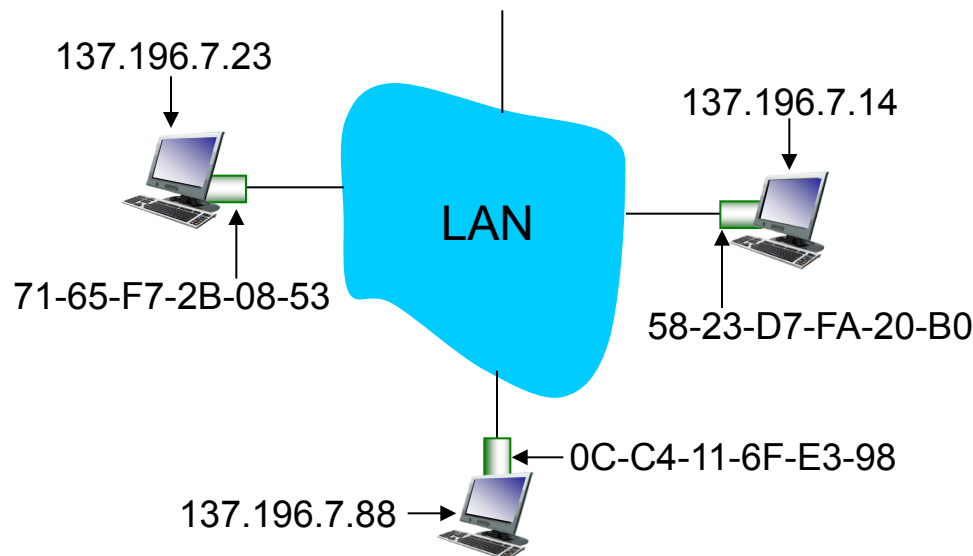
# LAN addresses (more)

- ❖ MAC address allocation administered by IEEE
- ❖ manufacturer of MAC
- ❖
  - MA ber
  - IP address: like postal address

# ARP: address resolution protocol

*Question:* how to determine interface's MAC address, knowing its IP address?

*ARP table:* each IP node , router) on LAN



- for some LAN nodes:
- < IP address; MAC address; TTL >
- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)

# ARP protocol: same LAN

- ❖ A wants to send datagram to B
  - B's MAC address not in A's ARP table.
- ❖ A **broadcasts** A

- ❖ A caches (saves) IP-to-MAC address pair its ARP table until S

addre

- dest MAC address = FF-FF-FF-FF-FF-FF
- all nodes on LAN receive ARP query
- ❖ B receives ARP packet, replies to A with its (B's) MAC address
  - frame sent to A's MAC address (unicast)

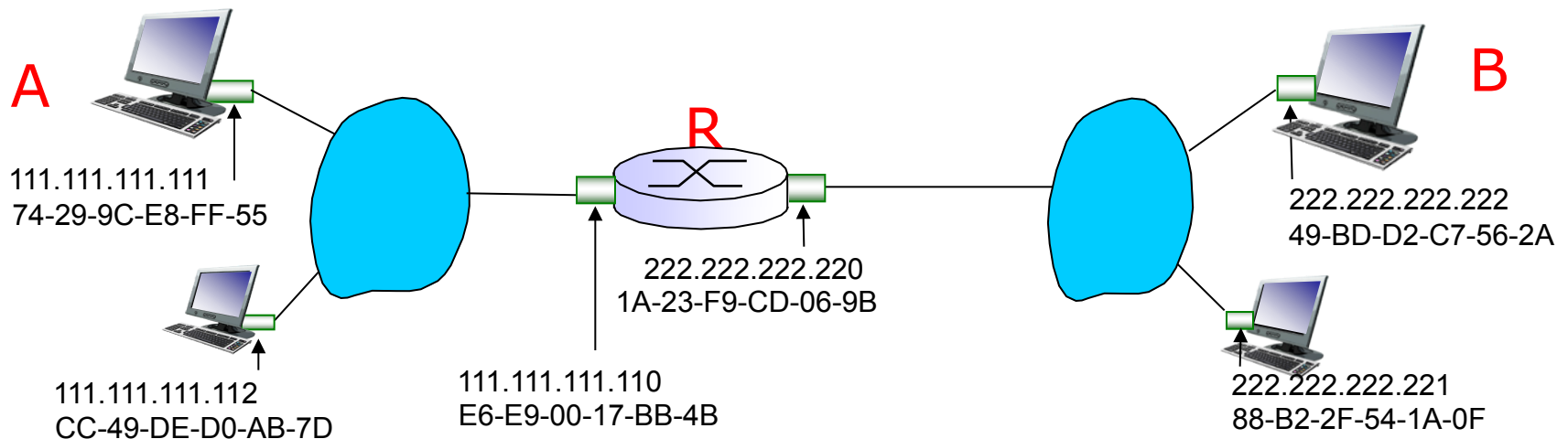
information  
that times out (goes away) unless refreshed

- ❖ ARP is "plug-and-play":
  - nodes create their ARP tables *without intervention from net administrator*

# Addressing: routing to ~~another LAN~~

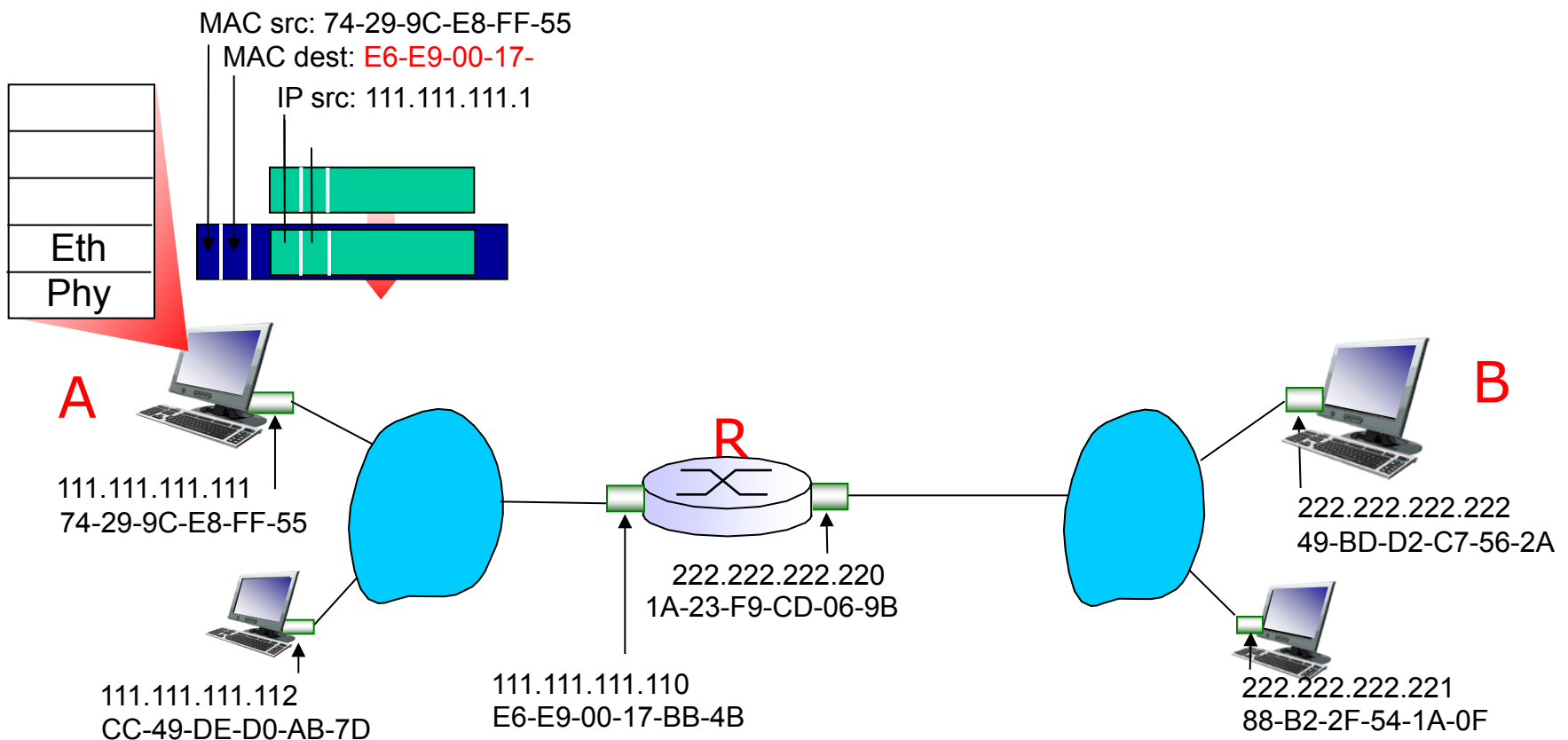
walkthrough: **send datagram from A to B via R**

- focus on addressing – at IP (datagram) and MAC layer (frame)
- assume A knows B's IP address
- assume A know first hop router, R



# Addressing: routing to ~~another LAN~~

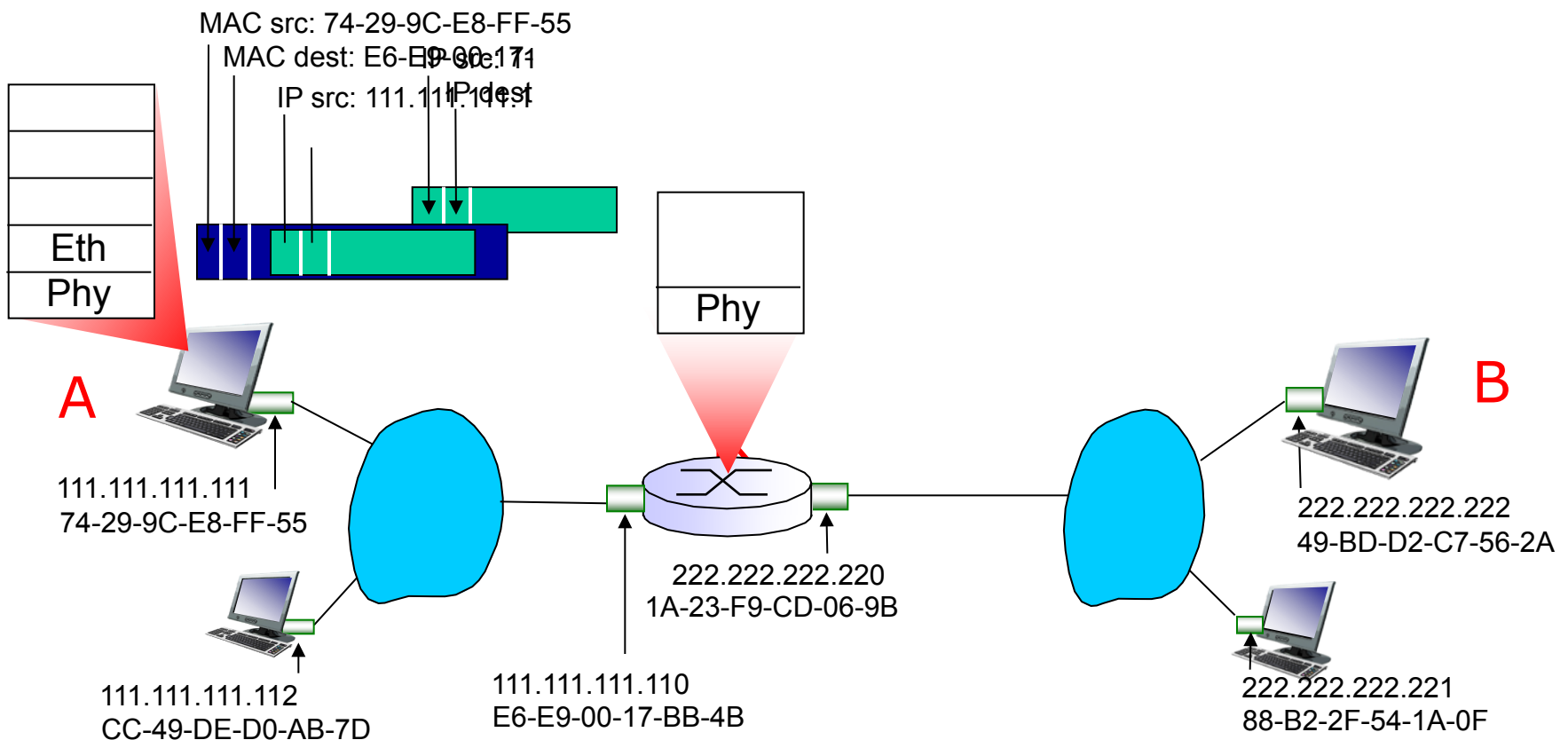
- ❖ A creates IP datagram with IP source A, destination B
- ❖ A creates link-layer frame with R's MAC address as dest, frame contains A-to-B IP datagram





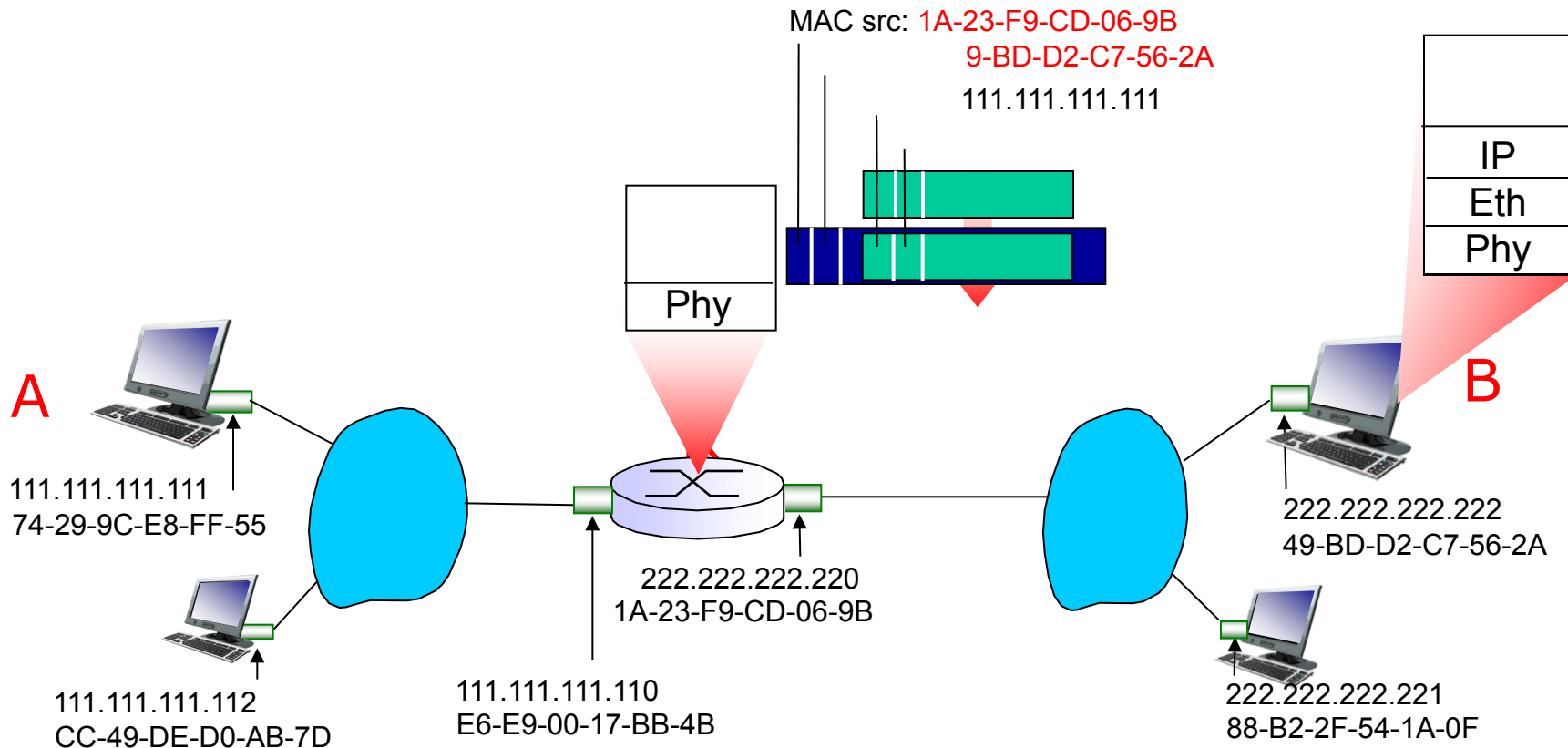
# Addressing: routing to ~~another LAN~~

- ❖ frame sent from A to R
- ❖ frame received at R, datagram removed, passed up to IP



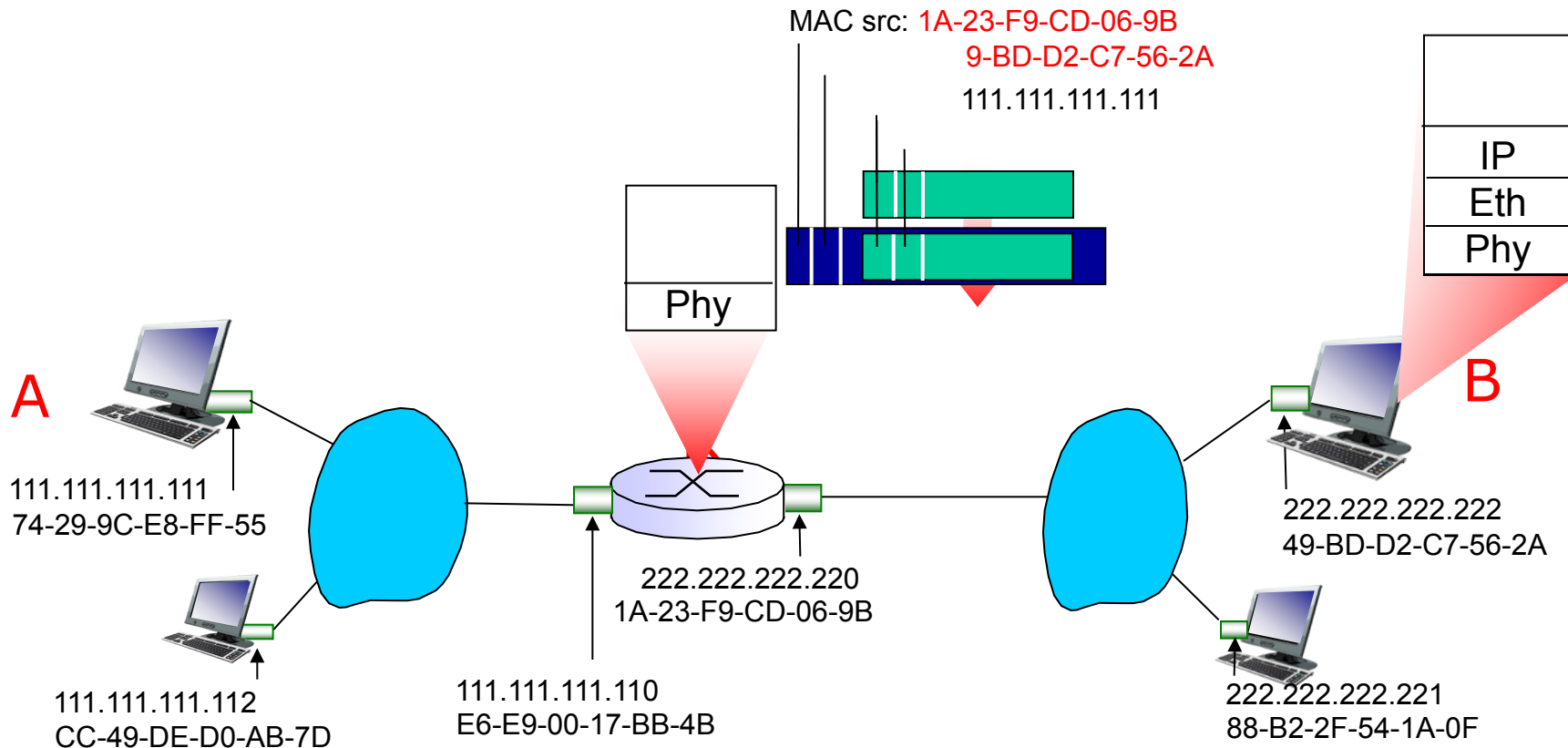
# Addressing: routing to another LAN

- ❖ R forwards datagram with IP source A, destination B
- ❖ R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



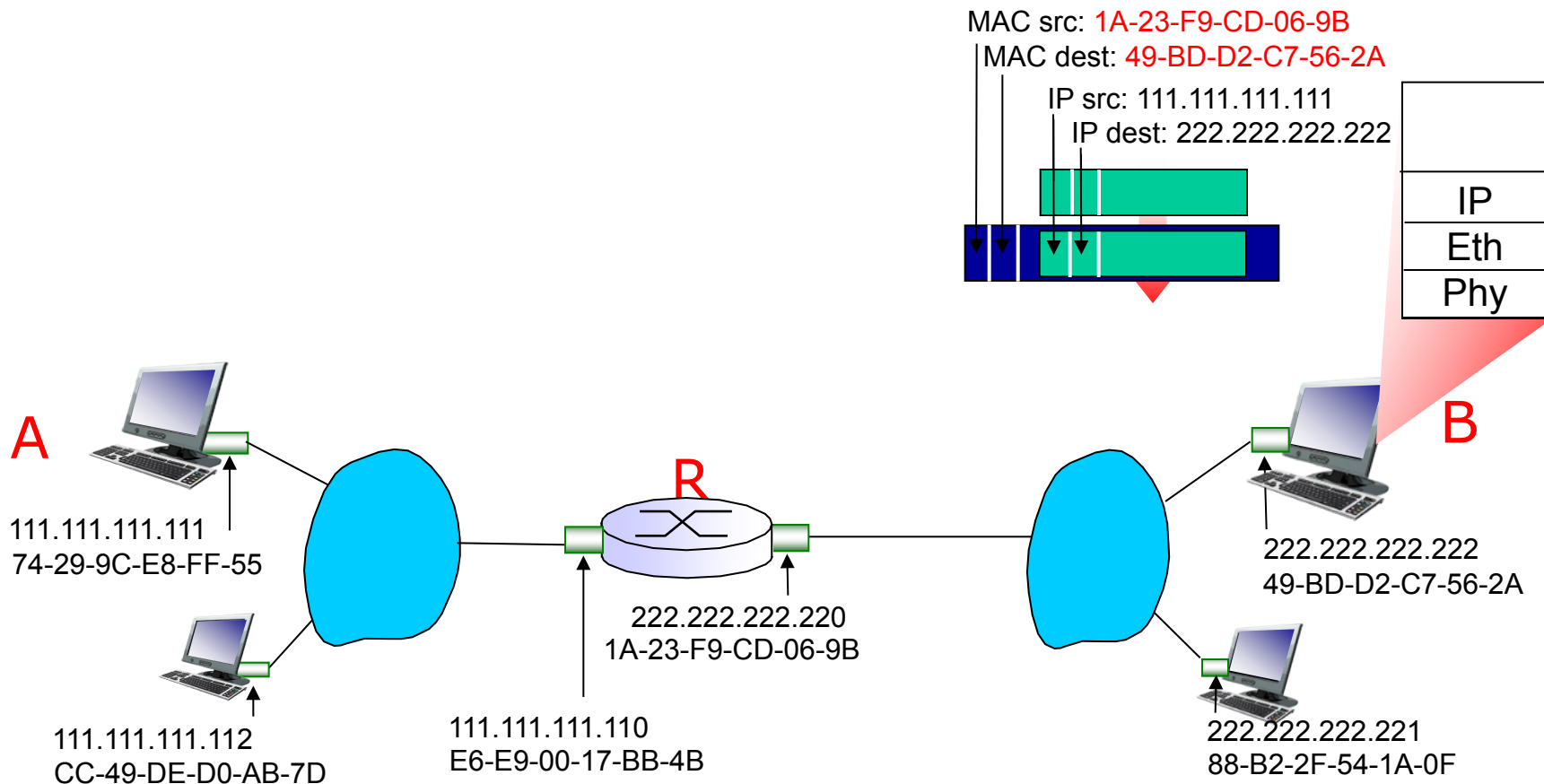
# Addressing: routing to another LAN

- ❖ R forwards datagram with IP source A, destination B
- ❖ R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



# Addressing: routing to another LAN

- ❖ R forwards datagram with IP source A, destination B
- ❖ R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot

5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

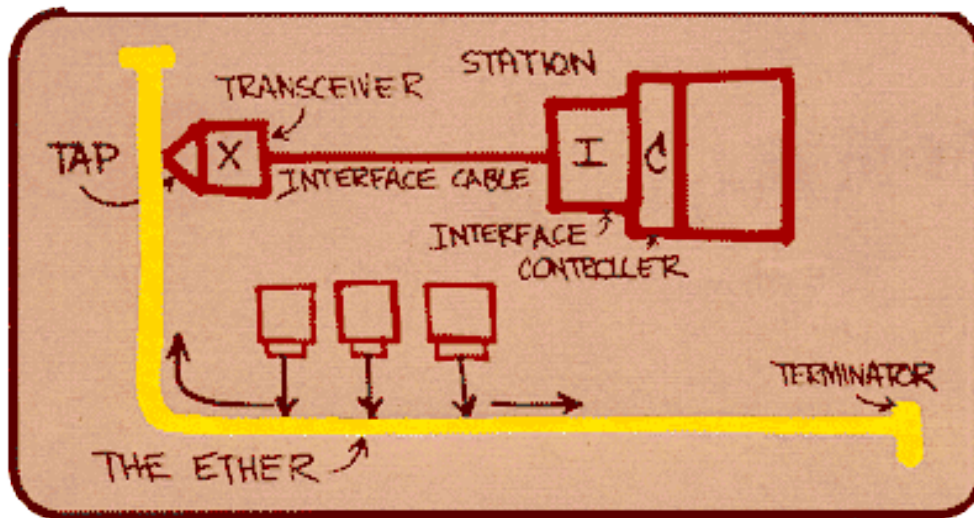
life  
of a web request

# Ethernet

“dominant” wired LAN technology:

- ❖ cheap \$20 for NIC
- ❖ first widely used LAN technology
- ❖ simpler, cheap

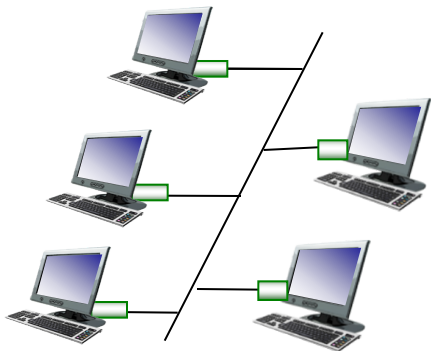
Ns and ATM



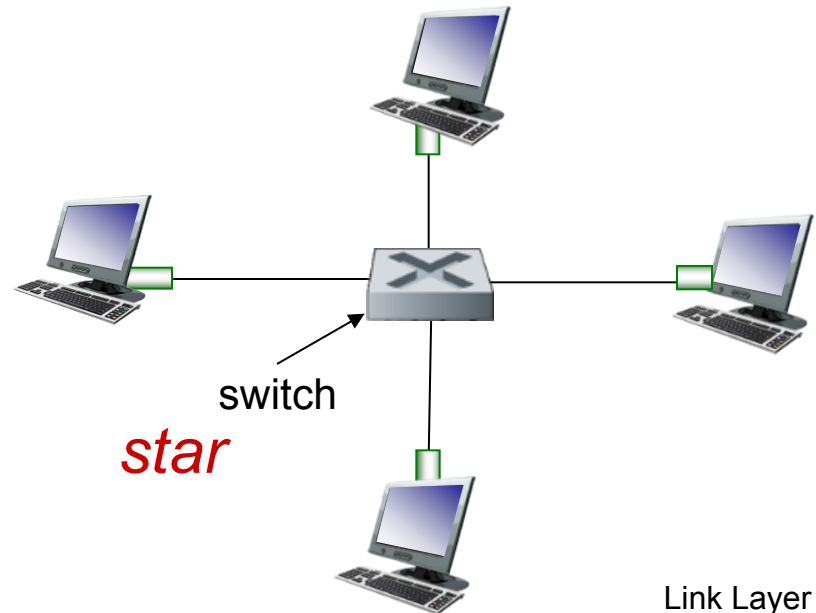
*Metcalfe's Ethernet sketch*

# Ethernet: physical topology

- ❖ *bus*: popular through mid 90s
  - all nodes in same collision domain (can collide with each other)
- ❖ *star*: prevails today
  - active *switch*



*bus*: coaxial cable



# Ethernet frame structure

sending adapter encapsulates IP  
datagram (or other network layer  
protocol pac

et frame



## *preamble:*

- ❖ 7 bytes with pattern 10101010 followed by one byte with pattern 10101011
- ❖ used to synchronize receiver, sender clock rates



# Ethernet frame structure

(more)

- ❖ **addresses:** 6 byte source, destination MAC addresses
  - if adapter receives frame with matching destination a broadcast address
  - oth
- ❖ **type:** indicates higher layer protocol (mostly IP but others possible, e.g., Novell IPX, AppleTalk)
- ❖ **CRC:** cyclic redundancy check at receiver

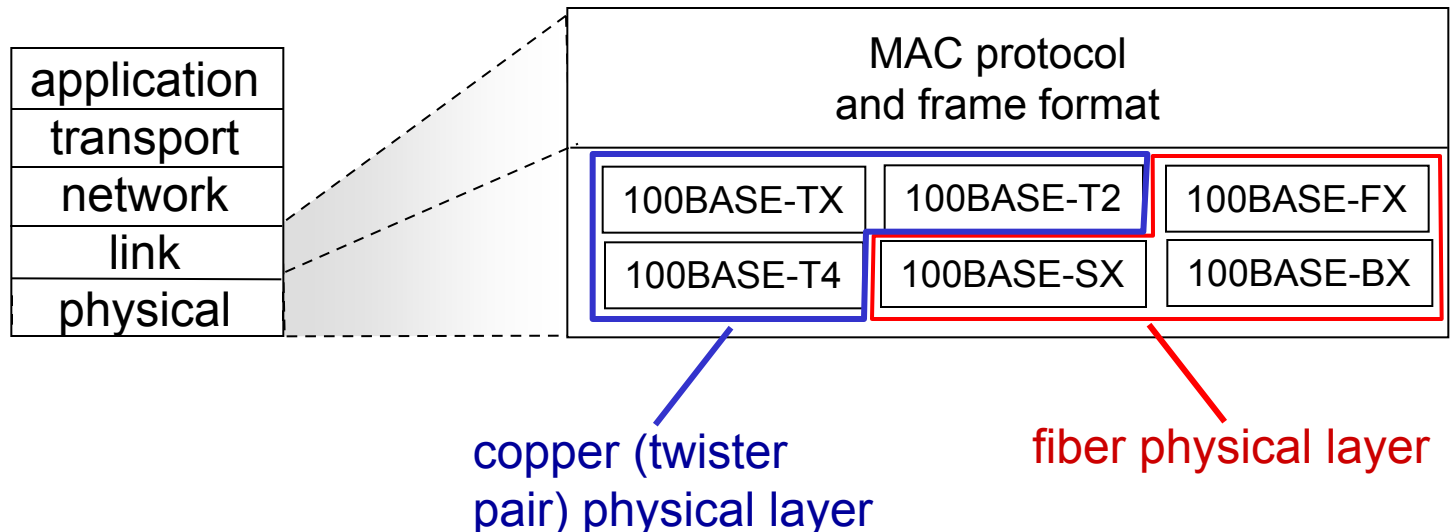


# Ethernet: unreliable, ~~connectionless~~

- ❖ *connectionless*: no handshaking between sending and receiving NICs
- ❖ *unreliable*: receiver doesn't send acks  
if  
initially (e.g.,  
TCP), otherwise dropped data lost
- ❖ Ethernet's MAC protocol: unslotted  
*CSMA/CD with binary backoff*

# 802.3 Ethernet standards: link & physical layers

- ❖ *many* different Ethernet standards
  - common MAC protocol and frame format
  - different speeds: 2 Mbps, 10 Mbps, 100 Mbps, 1Gbps



# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot

5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

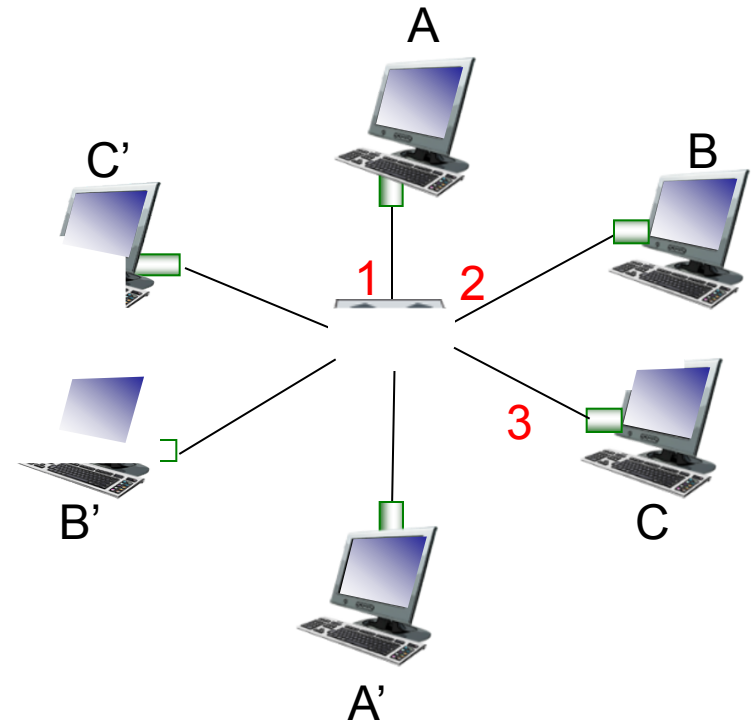
life  
of a web request

# Ethernet switch

- ❖ link-layer device: takes an *active* role
  - store, forward Ethernet frames
  - examine incoming frame's MAC address, *s* and frame to
- us t
- ❖ *transparent*
  - hosts are unaware of presence of switches
- ❖ *plug-and-play, self-learning*
  - switches do not need to be configured

# Switch: *multiple* simultaneous transmissions

- ❖ hosts have dedicated, direct connection to switch
- ❖ switches buffer
- ❖ but no duplex
  - each link is its own collision domain
- ❖ *switching*: A-to-A' and B-to-B' can transmit simultaneously, without collisions



switch with six interfaces  
(1,2,3,4,5,6)

# Switch forwarding table

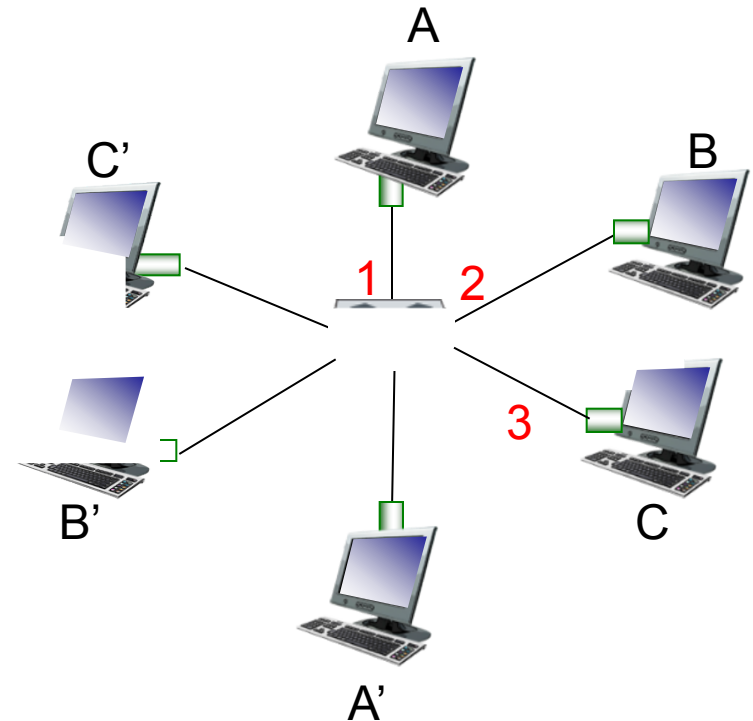
Q: how does switch know  
A' reachable via interface  
4, B' reachable via  
interface 5?

*entry*

- (MAC address of host,  
interface to reach host,  
time stamp)

Q: how are entries  
created, maintained in  
switch table?

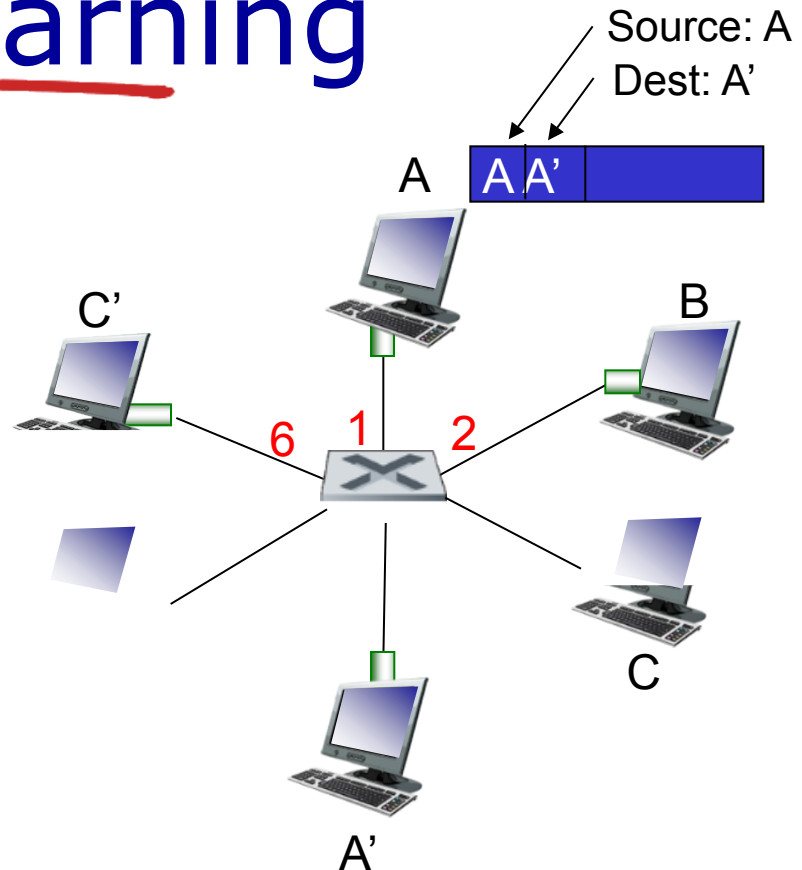
- something like a routing  
protocol?



switch with six interfaces  
(1,2,3,4,5,6)

# Switch: self-learning

- ❖ switch *learns* which hosts can be reached through which interfaces



of s  
incoming LAN  
segment

- records sender/location pair in switch

| MAC addr | interface | TTL |
|----------|-----------|-----|
| A        | 1         | 60  |
|          |           |     |
|          |           |     |
|          |           |     |

*Switch table  
(initially empty)*



# Switch: frame ~~filtering/forwarding~~

when frame received at switch:

1. record incoming link, MAC address of sending host

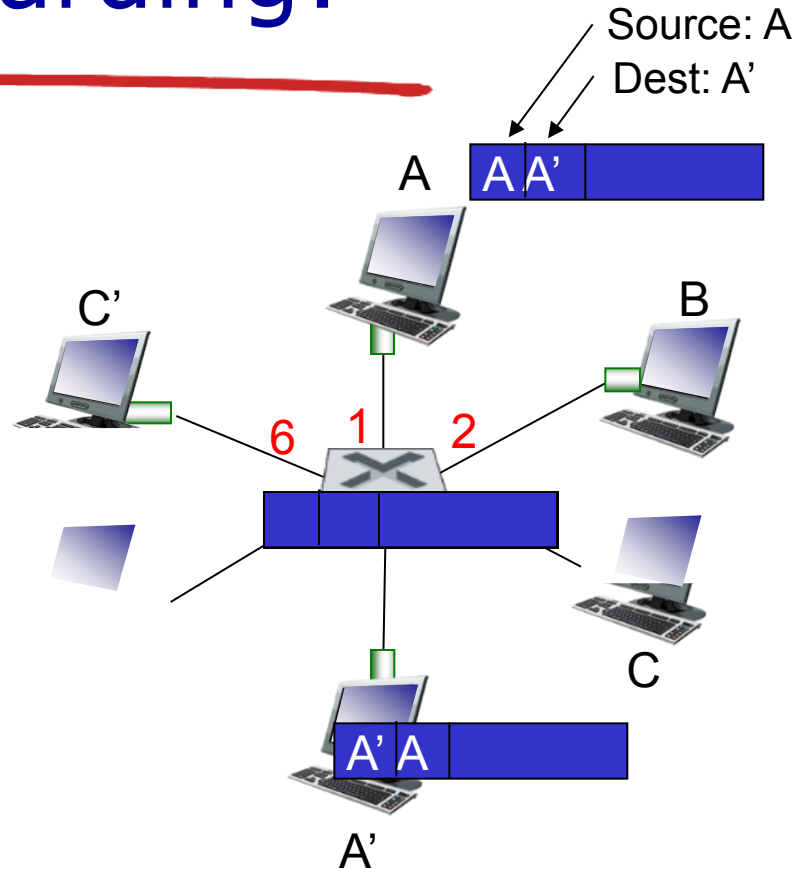
3. if  
    then {  
        if destination on segment from which frame  
        arrived  
            then drop frame  
            else forward frame on interface indicated  
by entry  
    }

- else flood /\* forward on all interfaces  
except arriving

# Self-learning, forwarding: ~~example~~

- ❖ frame destination, A', location *flood*
- ❖ unknown: destination A

*on just one link*

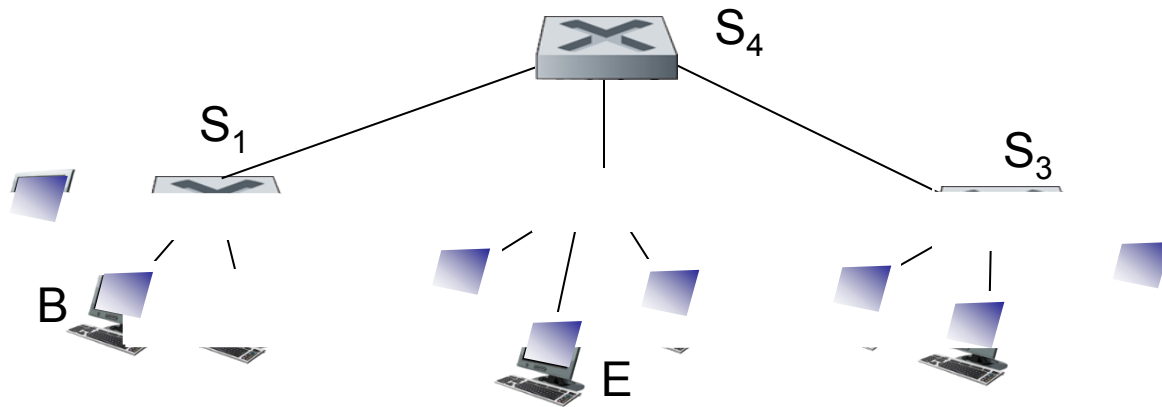


| MAC addr | interface | TTL |
|----------|-----------|-----|
| A        | 1         | 60  |
| A'       | 4         | 60  |

*switch table  
(initially empty)*

# Interconnecting switches

- ❖ switches can be connected together



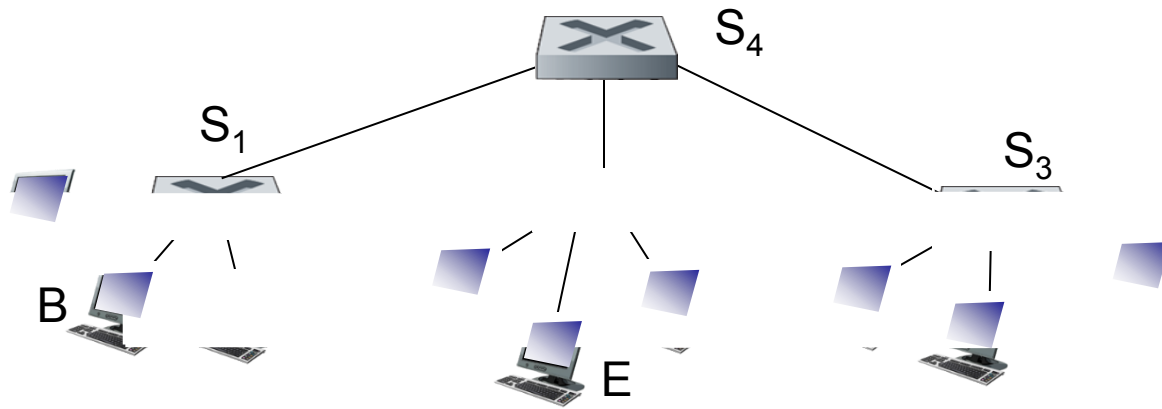
Q: sending from A to G - how does S<sub>1</sub> know to forward frame destined to F via S<sub>4</sub> and S<sub>3</sub>?

- ❖ A: self learning! (works *exactly* the same as in single-switch case!)

# Self-learning multi-switch

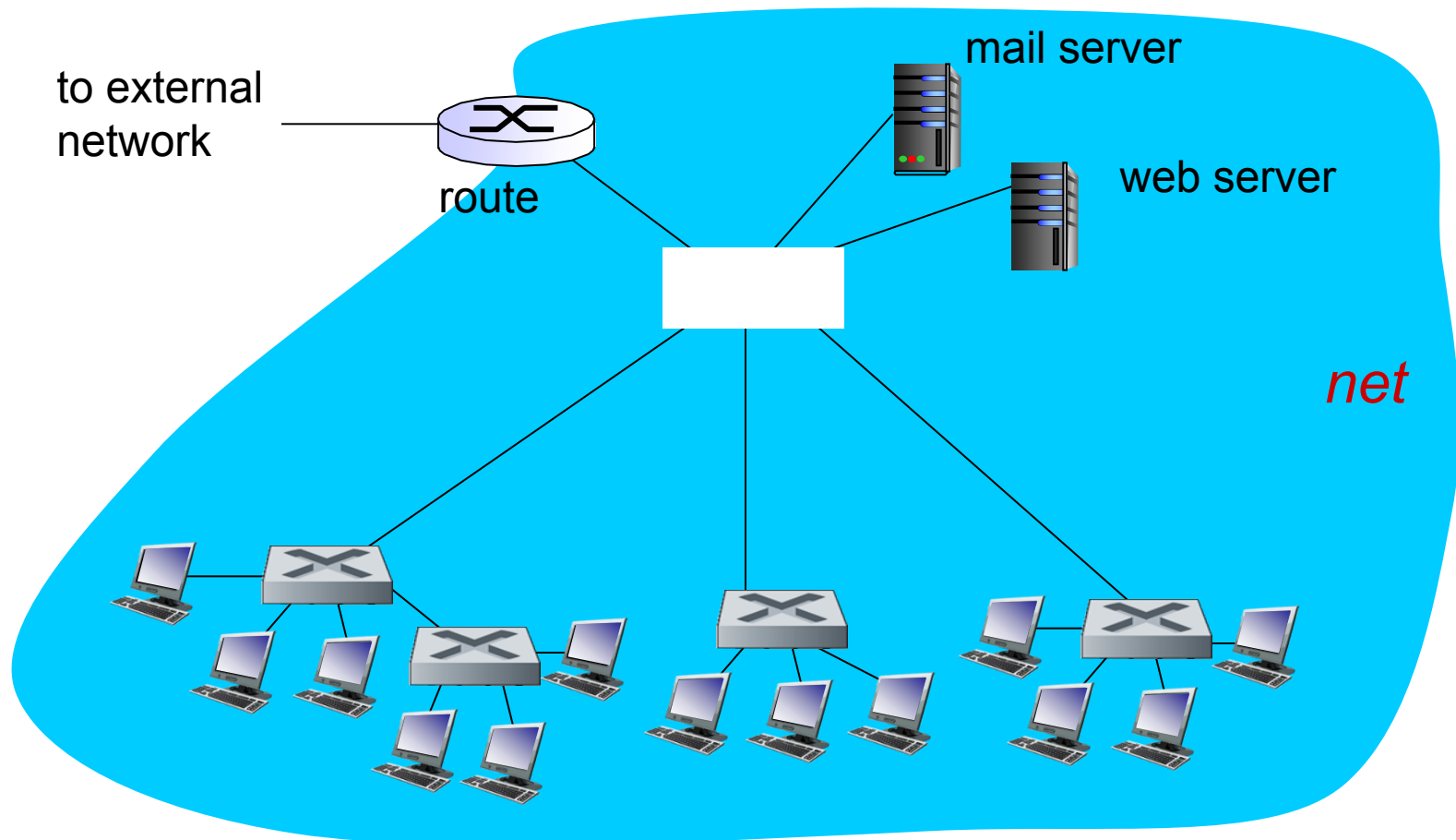
## ~~example~~

Suppose C sends frame to I, I responds to C



- ❖ Q: show switch tables and packet forwarding in S<sub>1</sub>, S<sub>2</sub>, S<sub>3</sub>, S<sub>4</sub>

# Institutional network



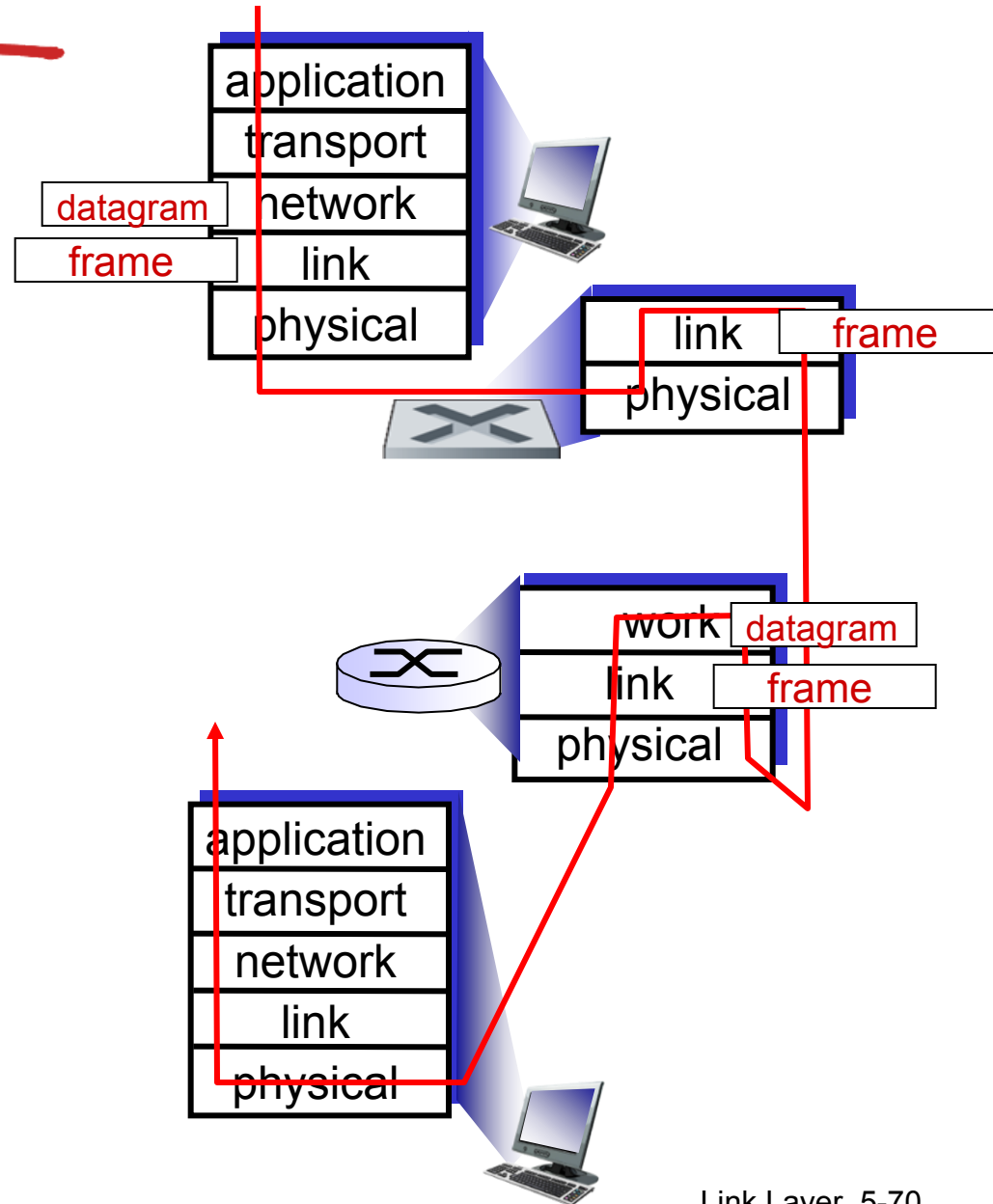
# Switches vs. routers

both are store-and-forward:

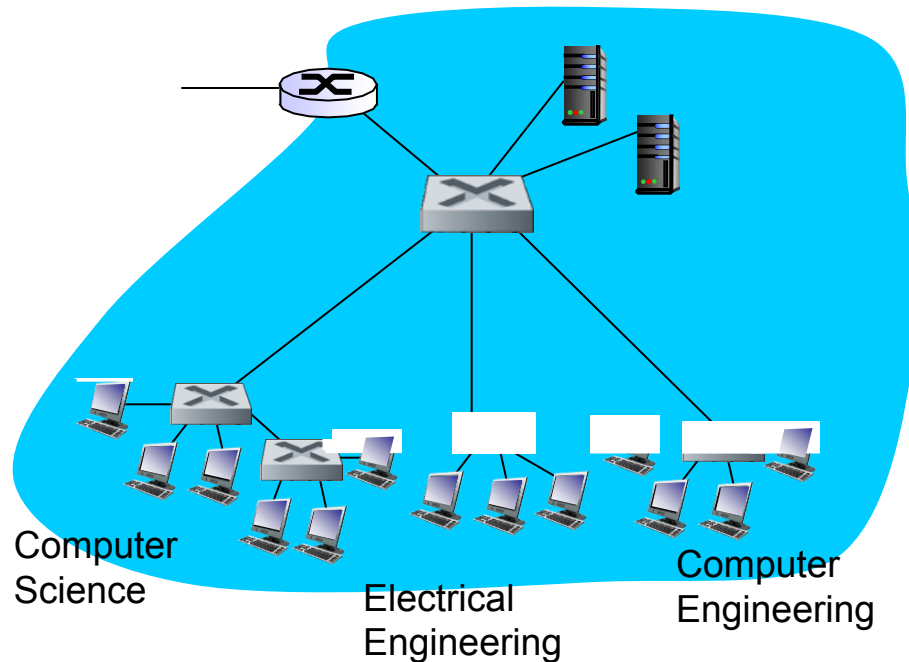
- **routers:** network-layer devices (examine network layer headers)
- **switches:** link-layer devices (examine link-layer headers)

both have forwarding tables:

- **routers:** compute tables using routing algorithms, IP addresses
- **switches:** learn forwarding tables using MAC addresses



# VLANs: motivation



*consider:*

- ❖ CS user moves office to EE, but wants connect to CS switch?

2  
broadcast traffic  
(ARP, DHCP,  
unknown location  
of destination MAC  
address) must  
cross entire LAN

- security/privacy,  
efficiency issues

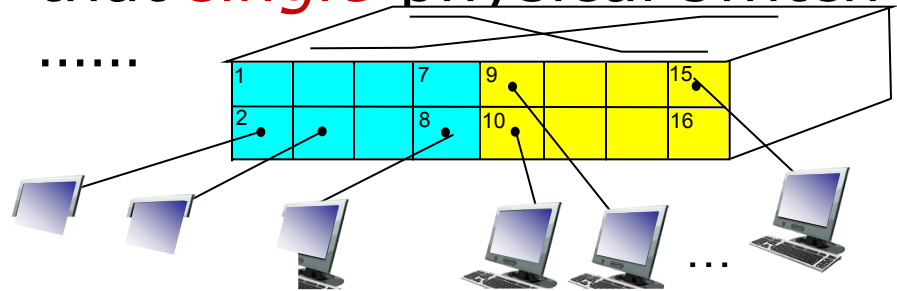
# VLANs

## *Virtual Local Area Network*

switch(es) supporting VLAN capabilities can

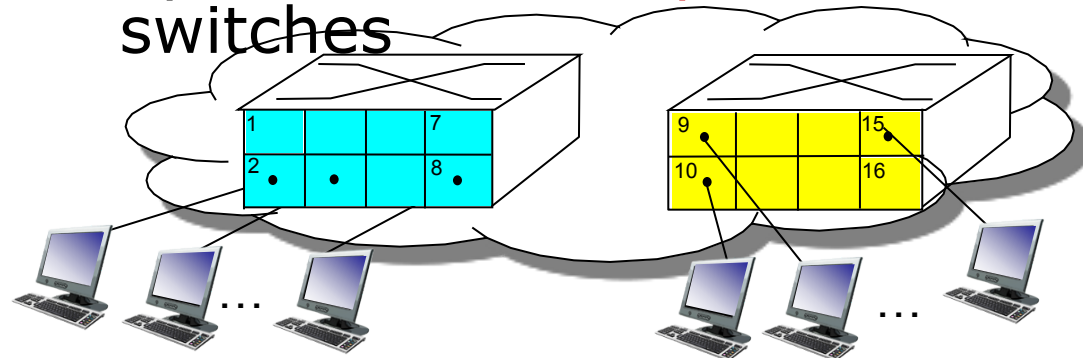
define multiple  
LANs over single  
physical LAN  
infrastructure.

**port-based VLAN:** switch ports grouped (by switch management software) so that *single* physical switch



N ports 9-15)

... operates as *multiple* virtual switches



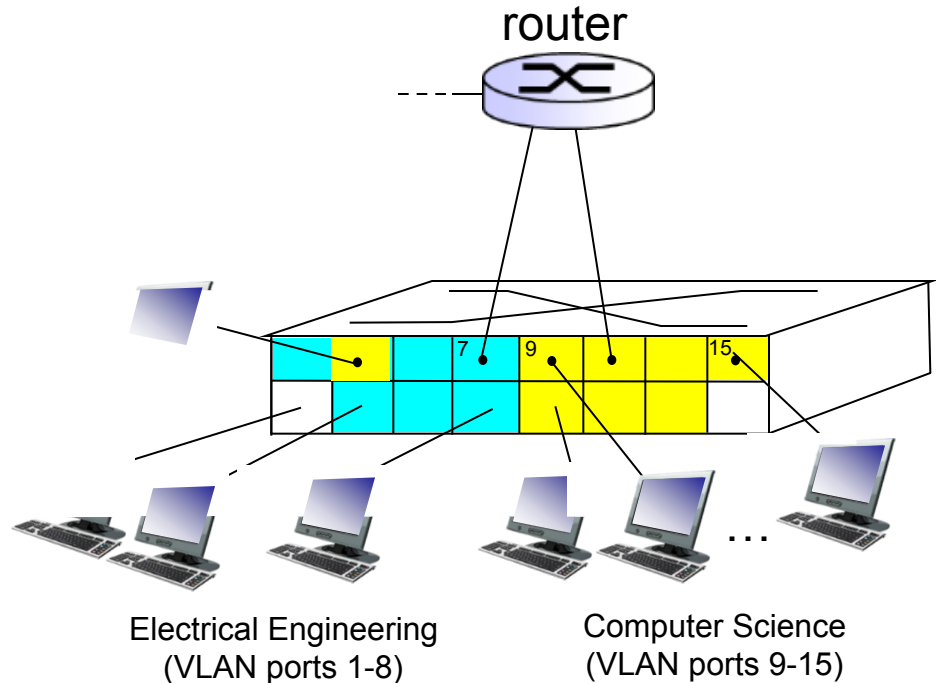
Electrical Engineering  
(VLAN ports 1-8)

Computer Science  
(VLAN ports 9-16)

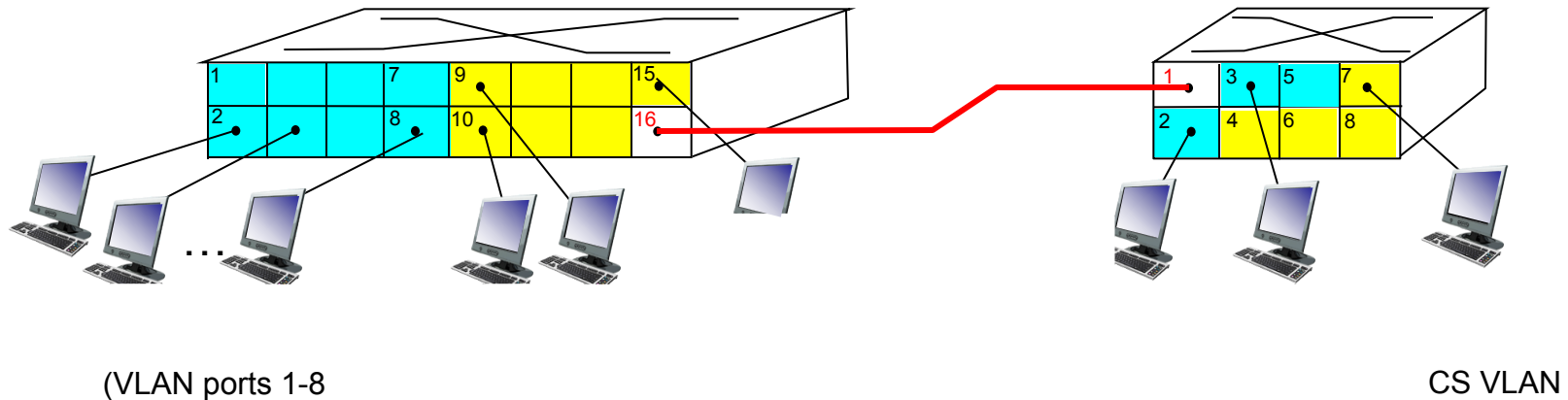


# Port-based VLAN

- ❖ **traffic isolation:** frames to/from ports 1-8 can *only* reach ports 1-8
  - can also define VLAN based on MAC addresses
- ❖ ports can dynamically assigned among VLANs
- ❖ **forwarding between VLANs:** done via routing (just as with separate switches)
  - in practice vendors sell combined switches plus routers

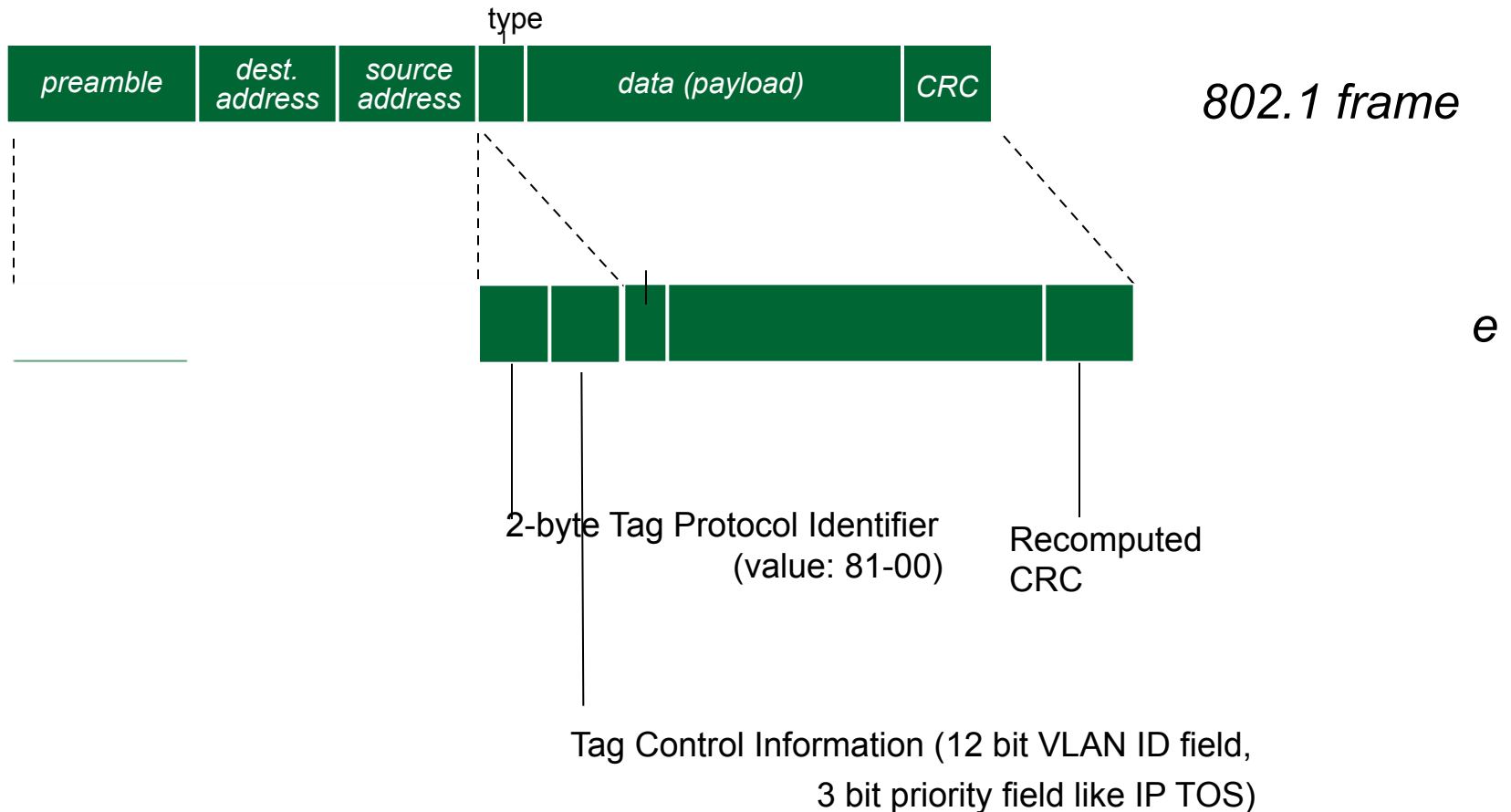


# VLANs spanning multiple switches



- ❖ **trunk port:** carries frames between VLANs defined over multiple physical switches
  - frames forwarded within VLAN between switches can't be vanilla 802.1 frames (must carry VLAN ID info)
  - 802.1q protocol adds/removed additional header fields for frames forwarded between trunk ports

# 802.1Q VLAN frame format



# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot

5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

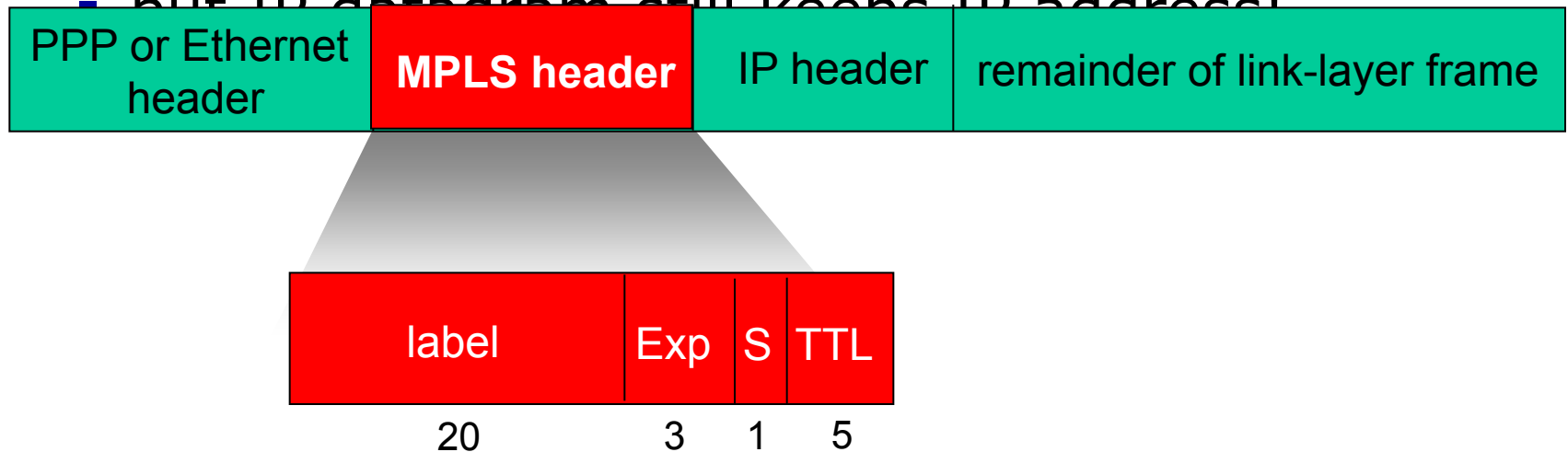
life  
of a web request

# Multiprotocol label switching (MPLS)

- ❖ initial goal: high-speed IP forwarding using fixed length label (instead of IP address)
  - fast lookup u identifier

app

- but IP datagram still keeps IP address



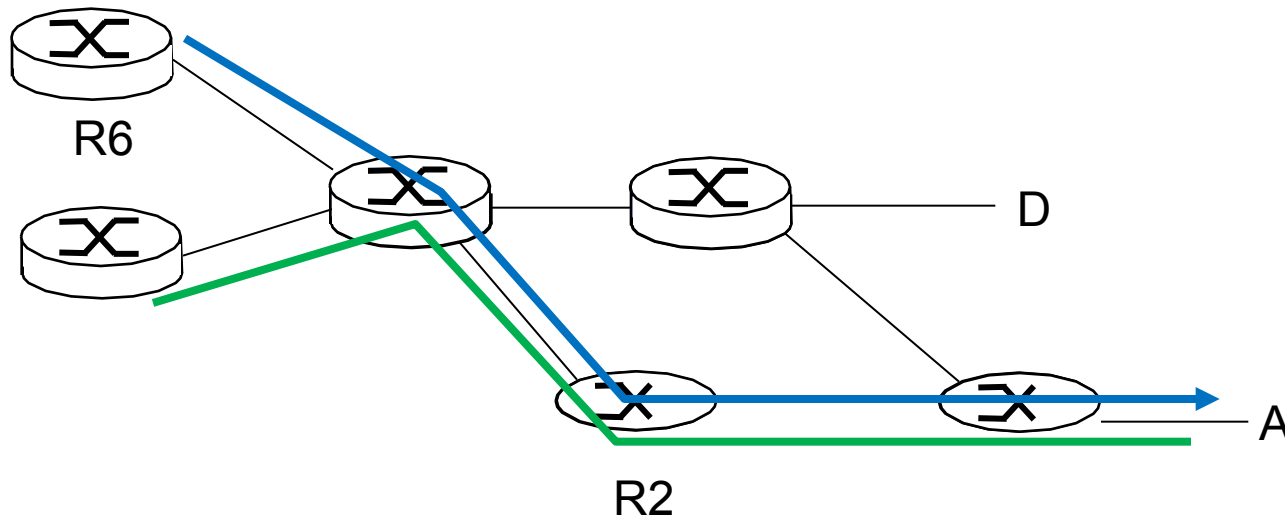
# MPLS capable routers

- ❖ a.k.a. label-switched router
- ❖ forward packets to outgoing interface based only on *don't inspect IP*

for

- ❖ *flexibility*: MPLS forwarding decisions can *differ* from those of IP
  - use destination *and* source addresses to route flows to same destination differently (traffic engineering)
  - re-route flows quickly if link fails: pre-computed backup paths (useful for VoIP)

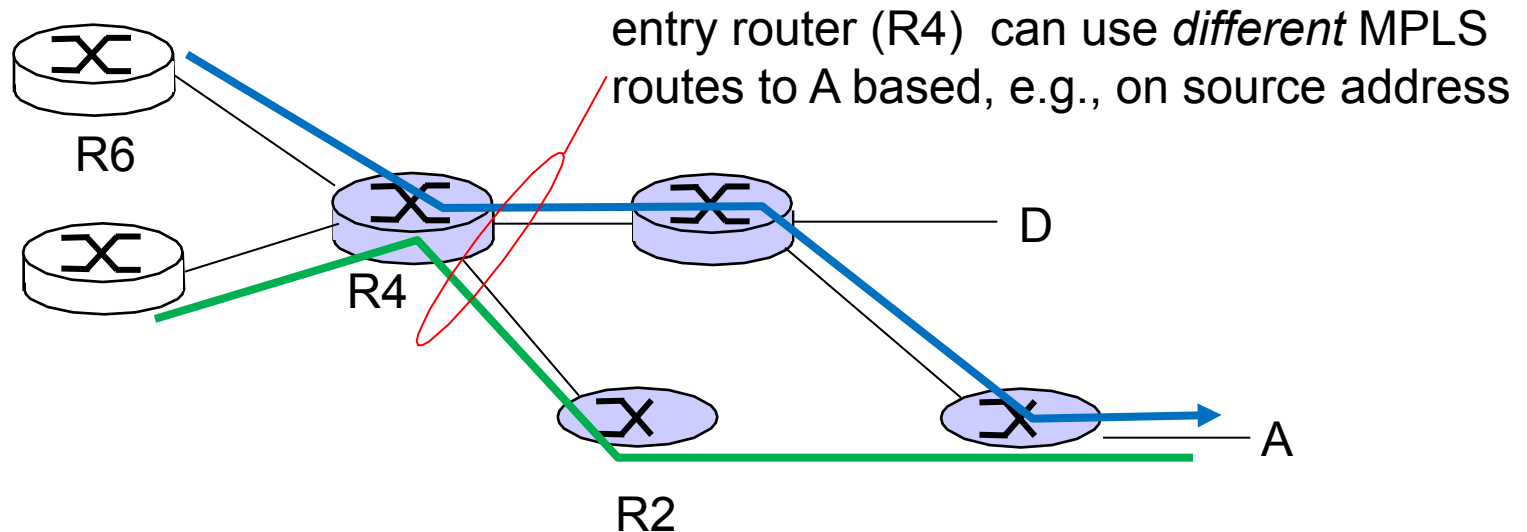
# MPLS versus IP paths



- ❖ *IP routing: path to destination determined by destination address alone*

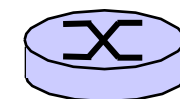


# MPLS versus IP paths



- ❖ **IP routing:** path to destination determined by destination address alone
- ❖ **MPLS routing:** path to destination can be based on source *and* dest. address
  - **fast reroute:** precompute backup routes in case of link failure

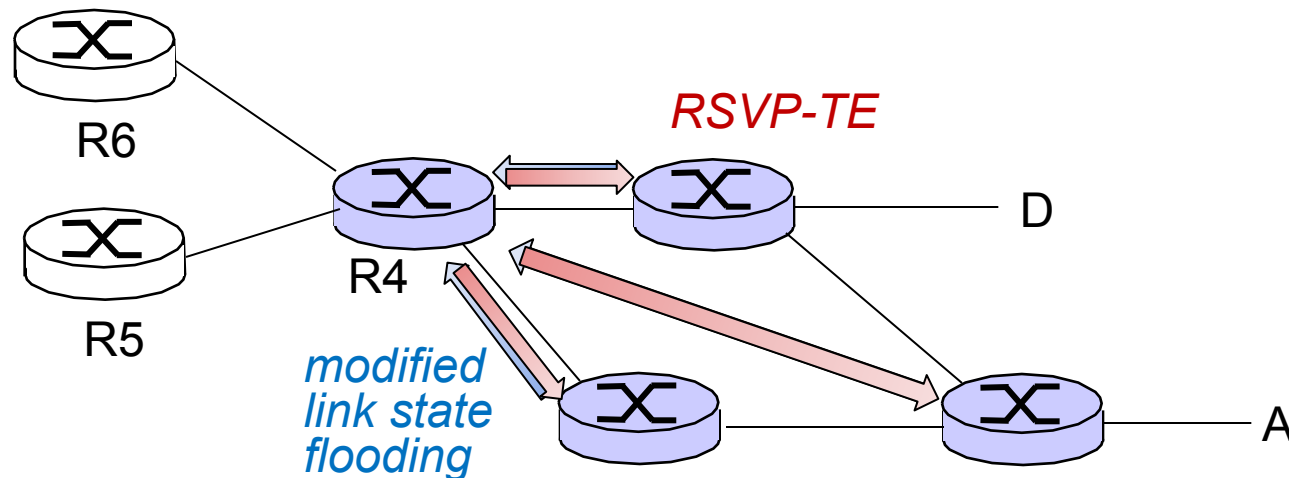
 IP-only router

 MPLS and IP router

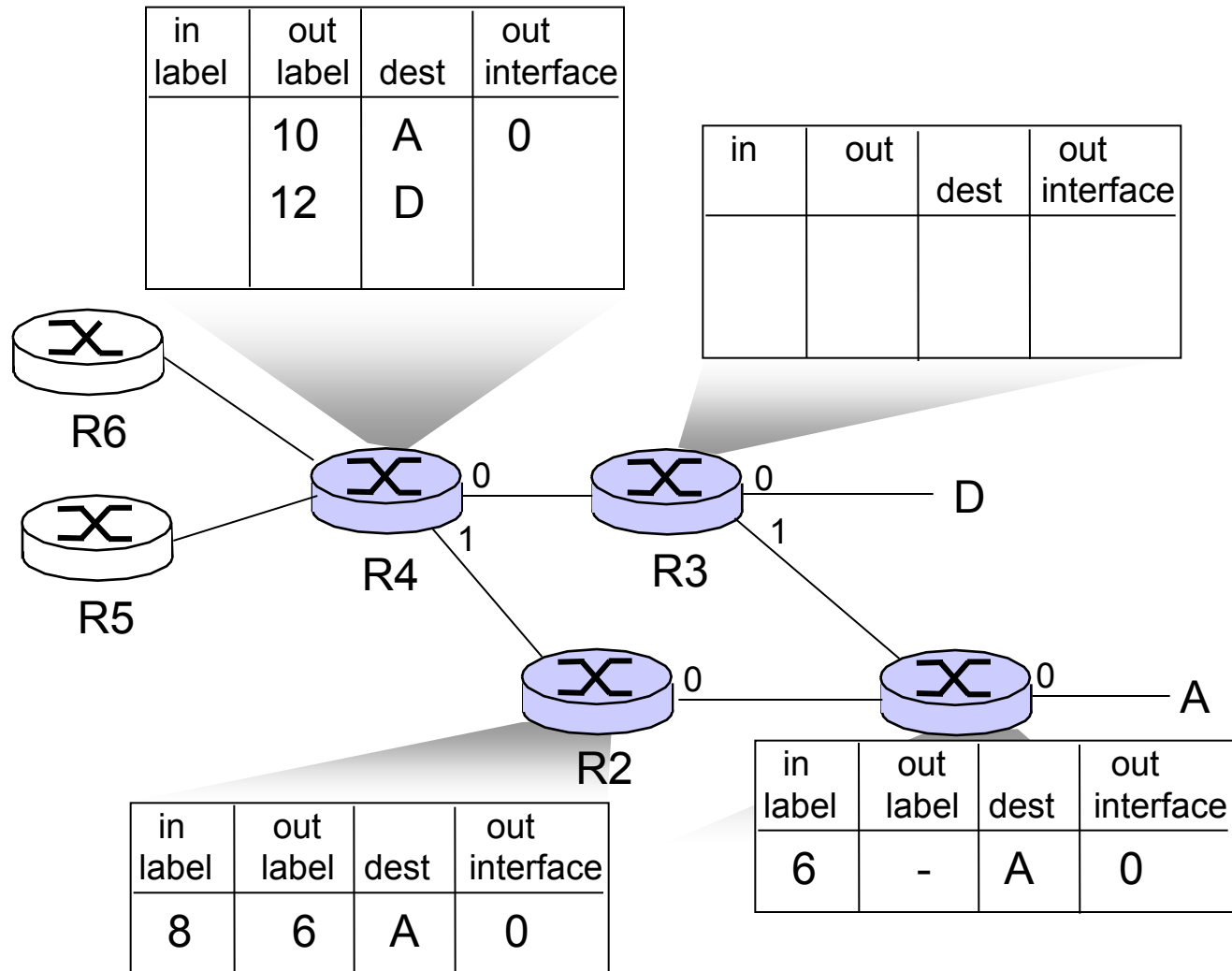


# MPLS signaling

- ❖ modify OSPF, IS-IS link-state flooding protocols to carry info used by MPLS routing,
    - e.g., Link State
  - ❖ entry, MPLS can
- of "Reserved" link  
VP-TE signaling



# MPLS forwarding tables



# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

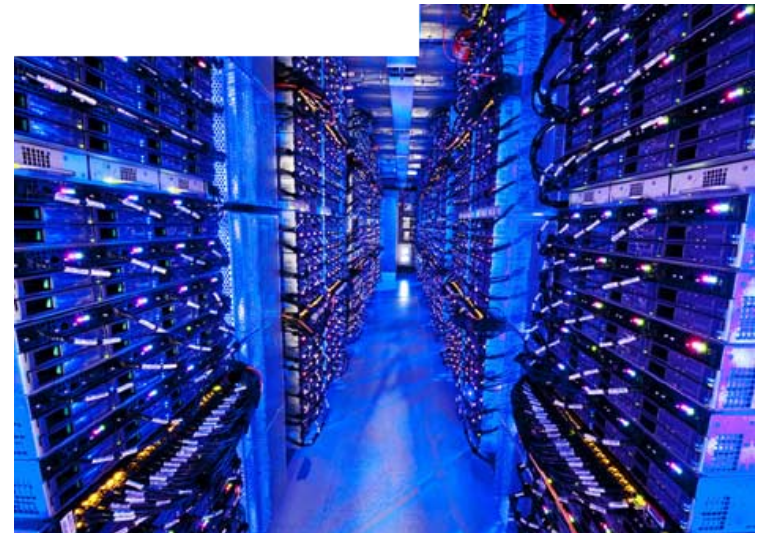
prot  
5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

life  
of a web request

# Data center networks

- ❖ 10's to 100's of thousands of hosts, often closely coupled, in close proximity:
  - e-business (e.g. Amazon)
  - content-serving (e.g. YouTube, Akamai, Apple, etc.)
- ❖ challenges
  - multiple applications, each serving massive numbers of clients
  - managing/balancing load, avoiding processing, networking, data bottlenecks

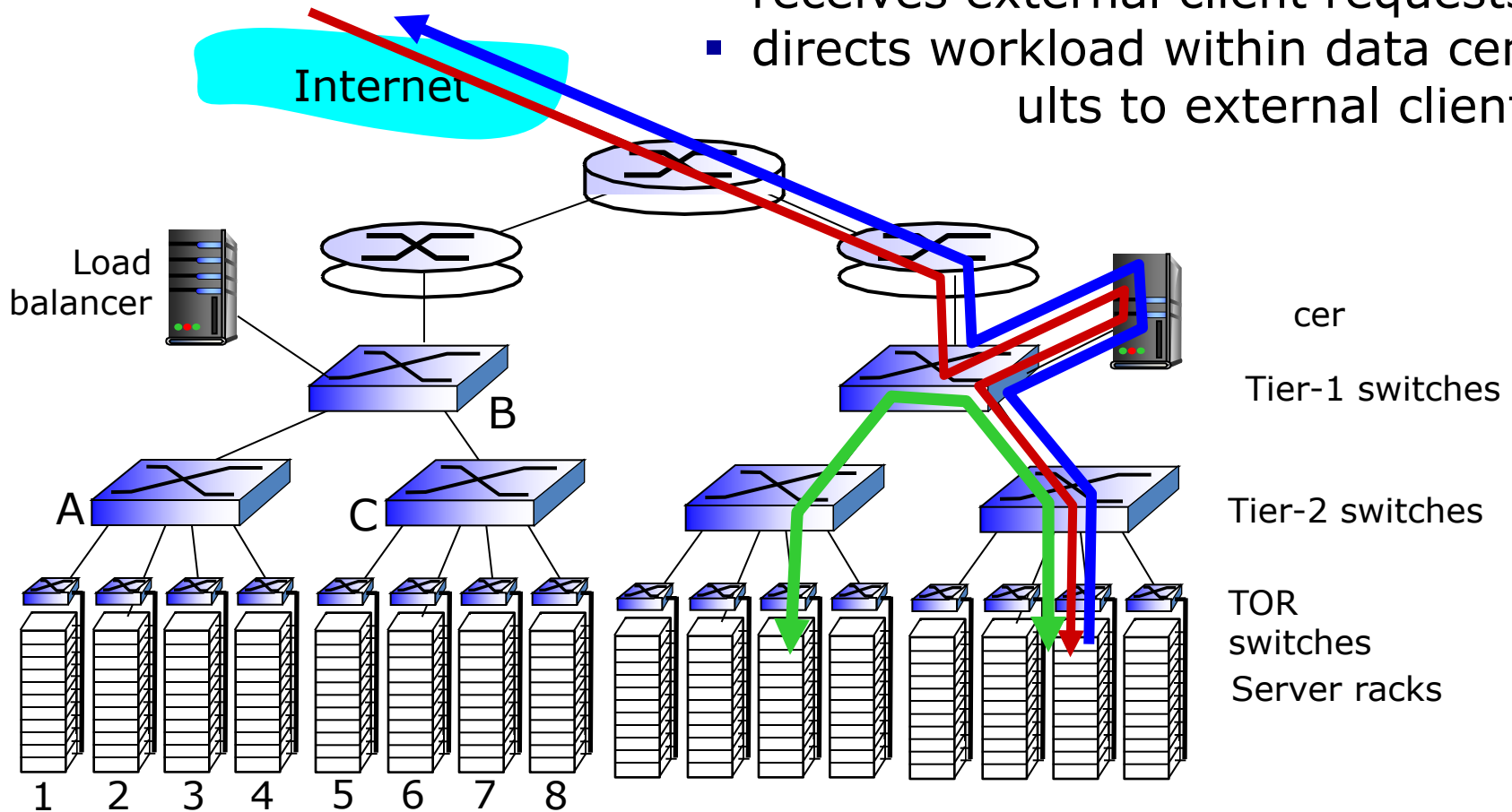


Inside a 40-ft Microsoft container, Chicago data center

# Data center networks

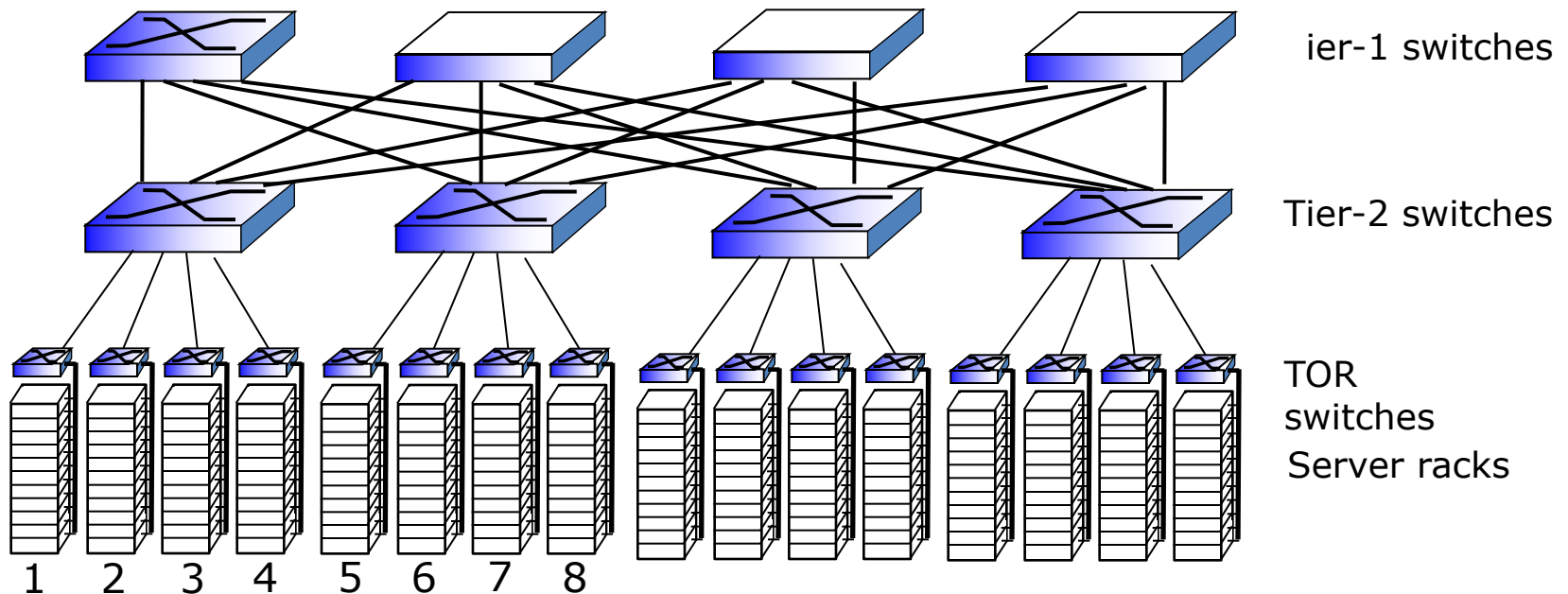
*load balancer: application-layer routing*

- receives external client requests
- directs workload within data center ults to external client m



# Data center networks

- ❖ rich interconnection among switches, racks:
  - increased throughput between racks (multiple routing paths possible)
  - increased reliability



# Link layer, LANs: outline

5.1 introduction,  
services

5.2 error detec

5.5 link  
virtualization:  
S

prot

5.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

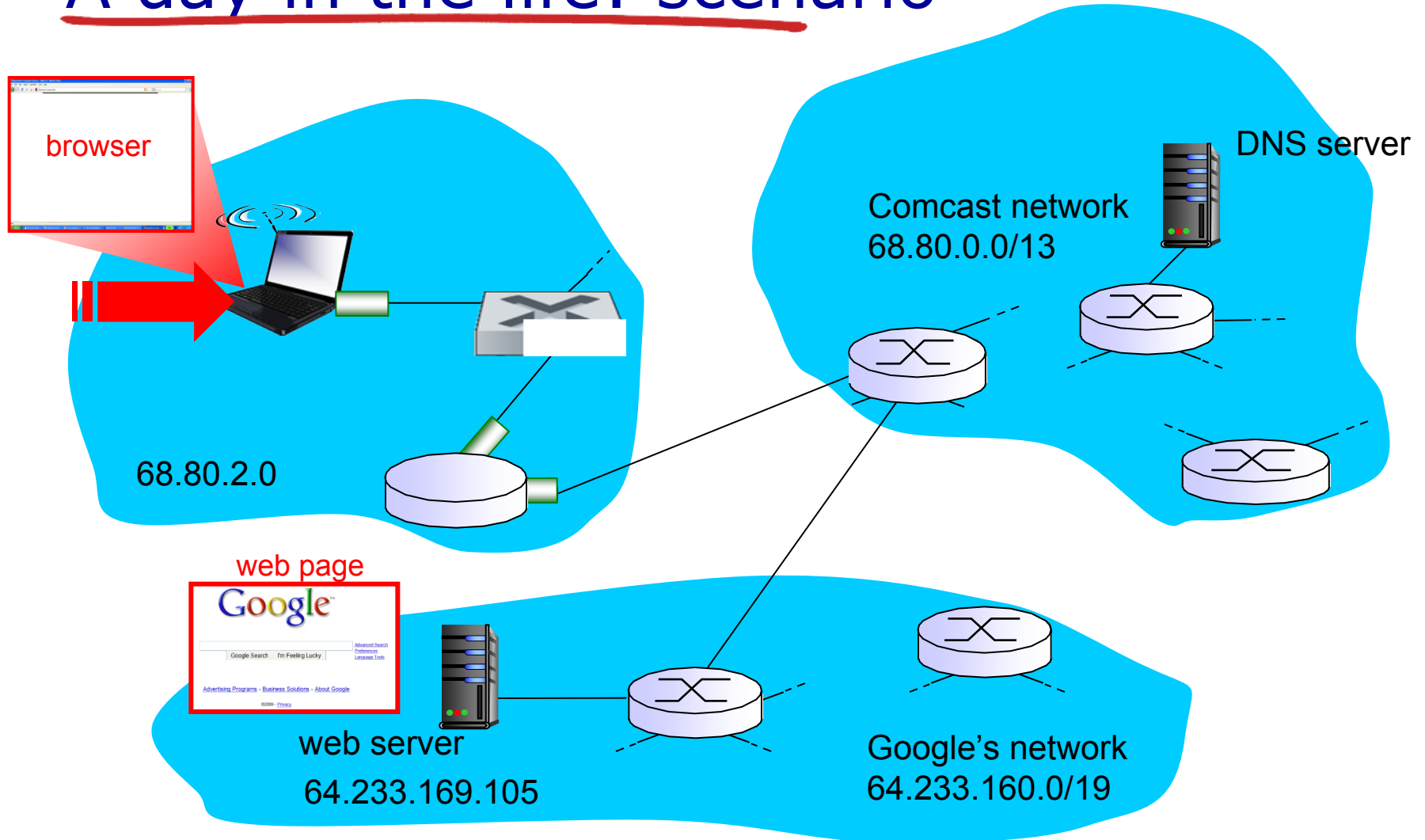
life  
of a web request

# *Synthesis:* a day in the life of a web request

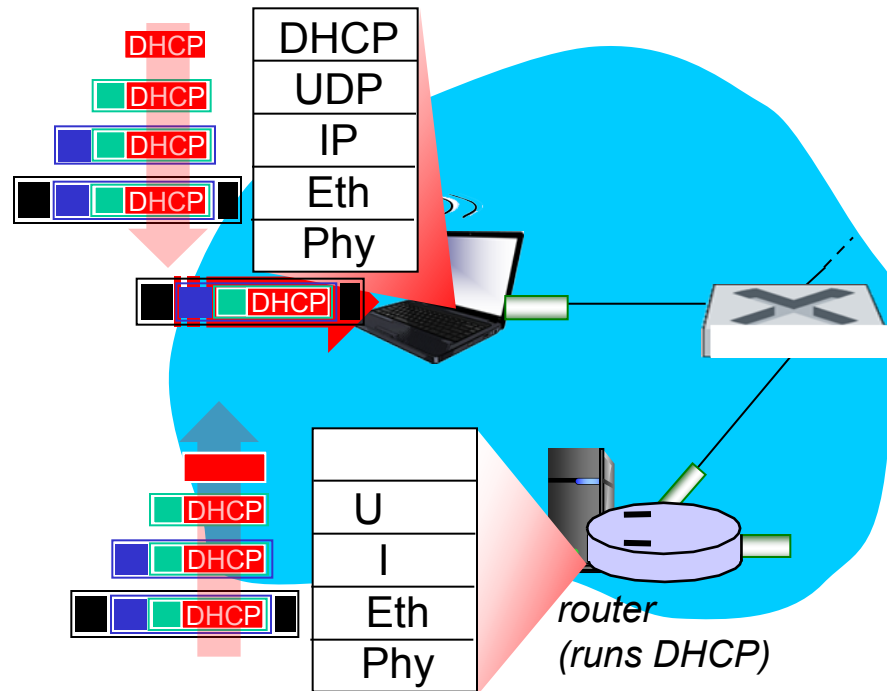
- ❖ journey down protocol stack complete!
  - application, transport, network, link
- ❖ putting-it-all-together: synthesis!
  - *goal:* identify tand protocols
  - *sce*  
campus network, requests/receives  
www.google.com



# A day in the life: scenario



# A day in the life... connecting to the Internet

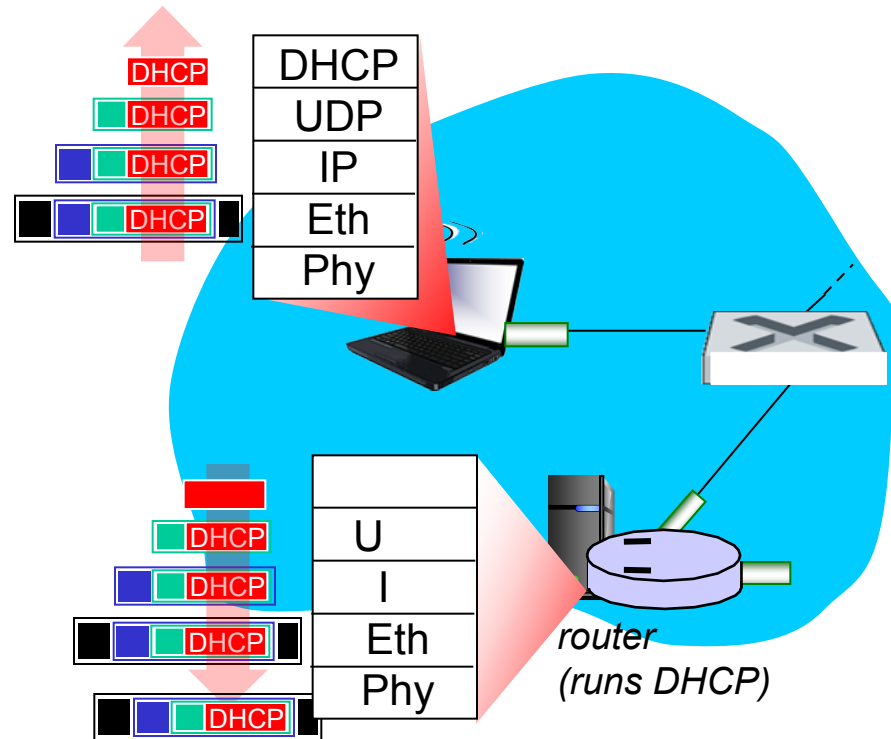


- ❖ connecting laptop needs to get its own IP address, addr of first-hop router, addr of DNS server: use **CP** request

in **802.3**

- ❖ Ethernet frame **broadcast** (dest: FFFFFFFFFFFFFFFF) on LAN, received at router
- ❖ running **DHCP** server
- ❖ Ethernet **demuxed** to IP demuxed, UDP demuxed to DHCP

# A day in the life... connecting to the Internet

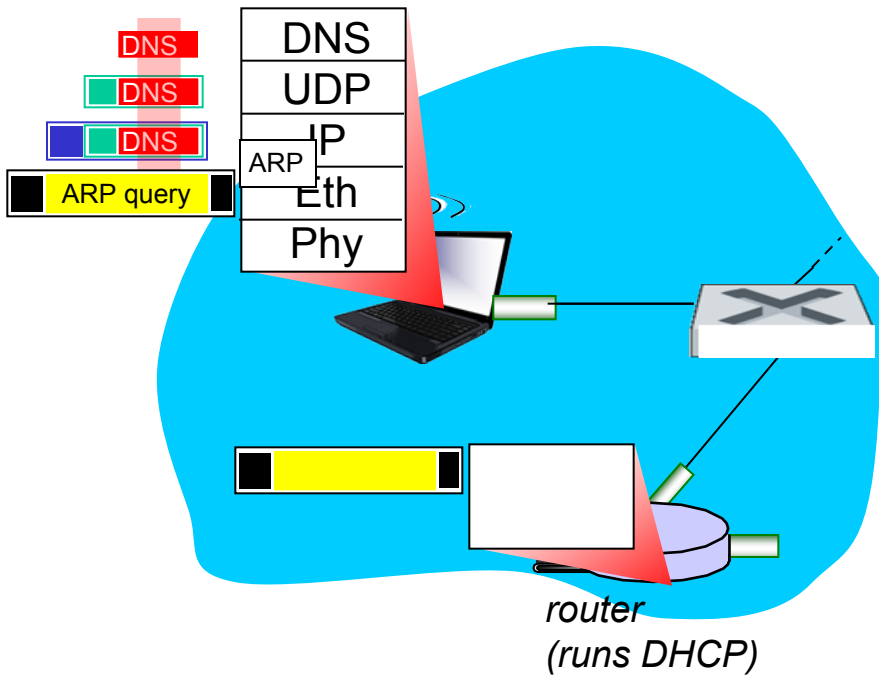


- ❖ DHCP server formulates **DHCP ACK** containing client's IP address, IP address of first-hop router for next hop, name & IP address of DNS server

- ❖ Client receives DHCP ACK reply

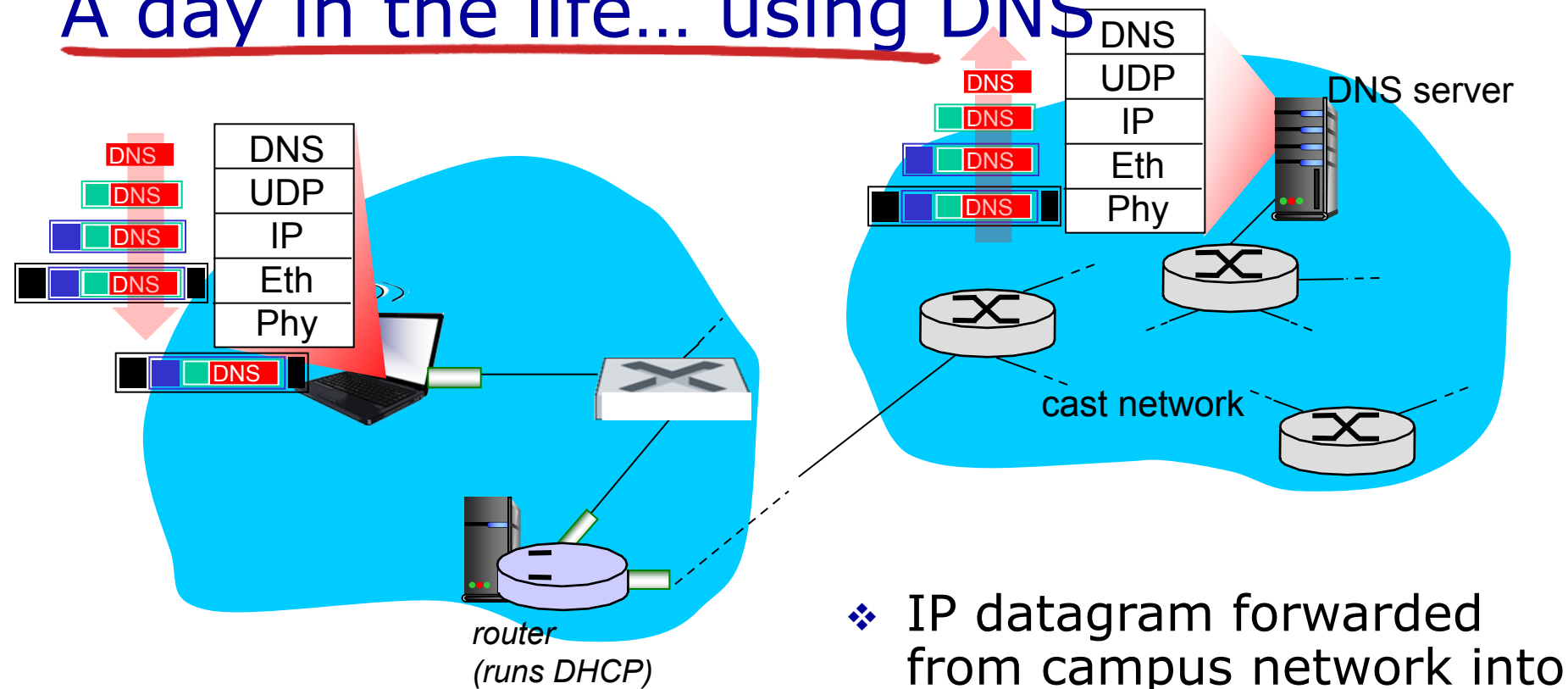
*Client now has IP address, knows name & addr of DNS server, IP address of its first-hop router*

# A day in the life... ARP (before DNS, before HTTP)



- ❖ before sending *HTTP* request, need IP address of `www.google.com`: *DNS*
- ❖ DNS query created,   
ulated in UDP,
- ❖ *ARP query* broadcast,   
interface: *ARP*   
received by router, which   
replies with *ARP reply*   
giving MAC address of   
router interface
- ❖ client now knows MAC   
address of first hop router,   
so can now send frame   
containing DNS query

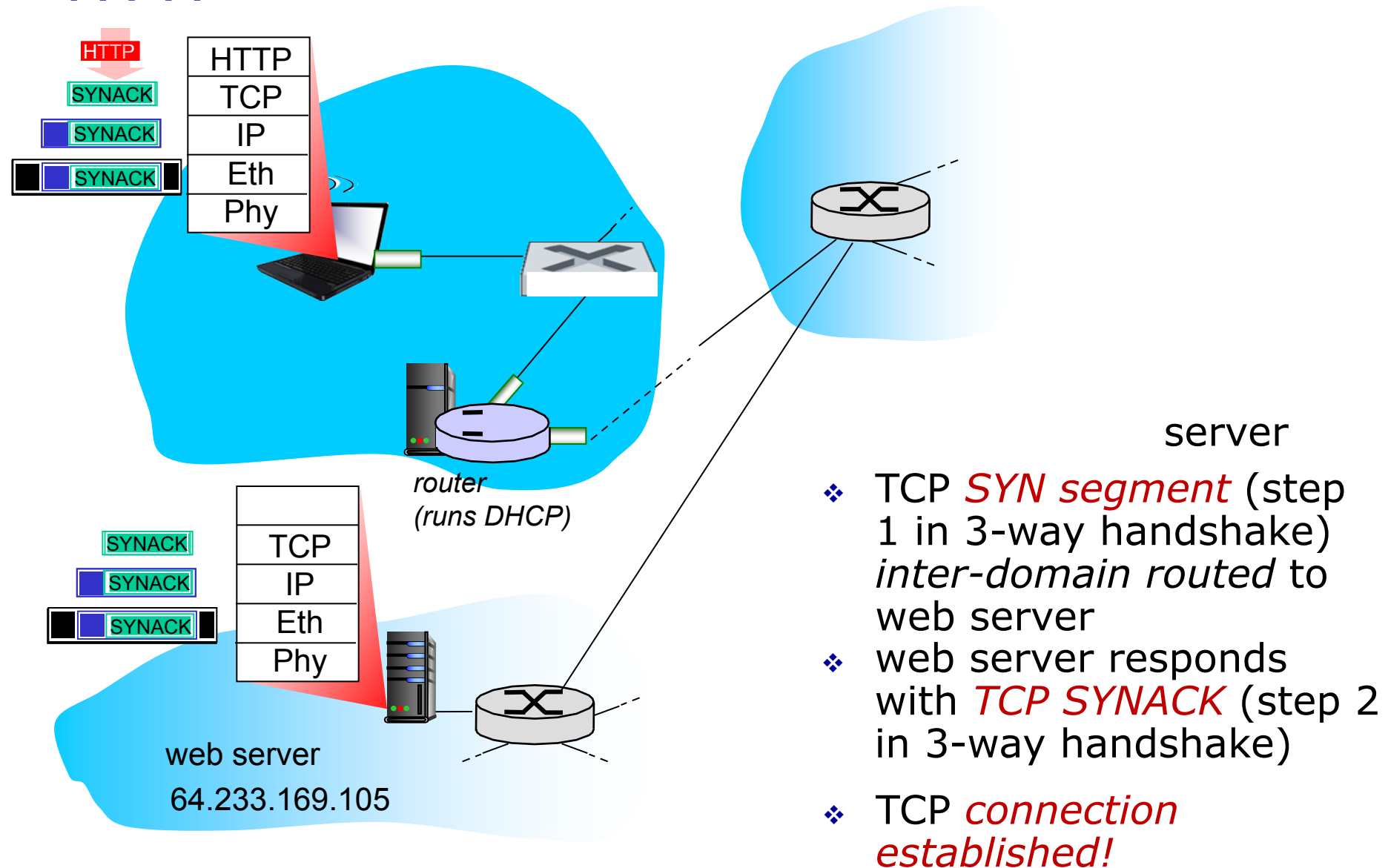
# A day in the life... using DNS



- ❖ IP datagram containing DNS query forwarded via LAN switch from client to 1<sup>st</sup> hop router

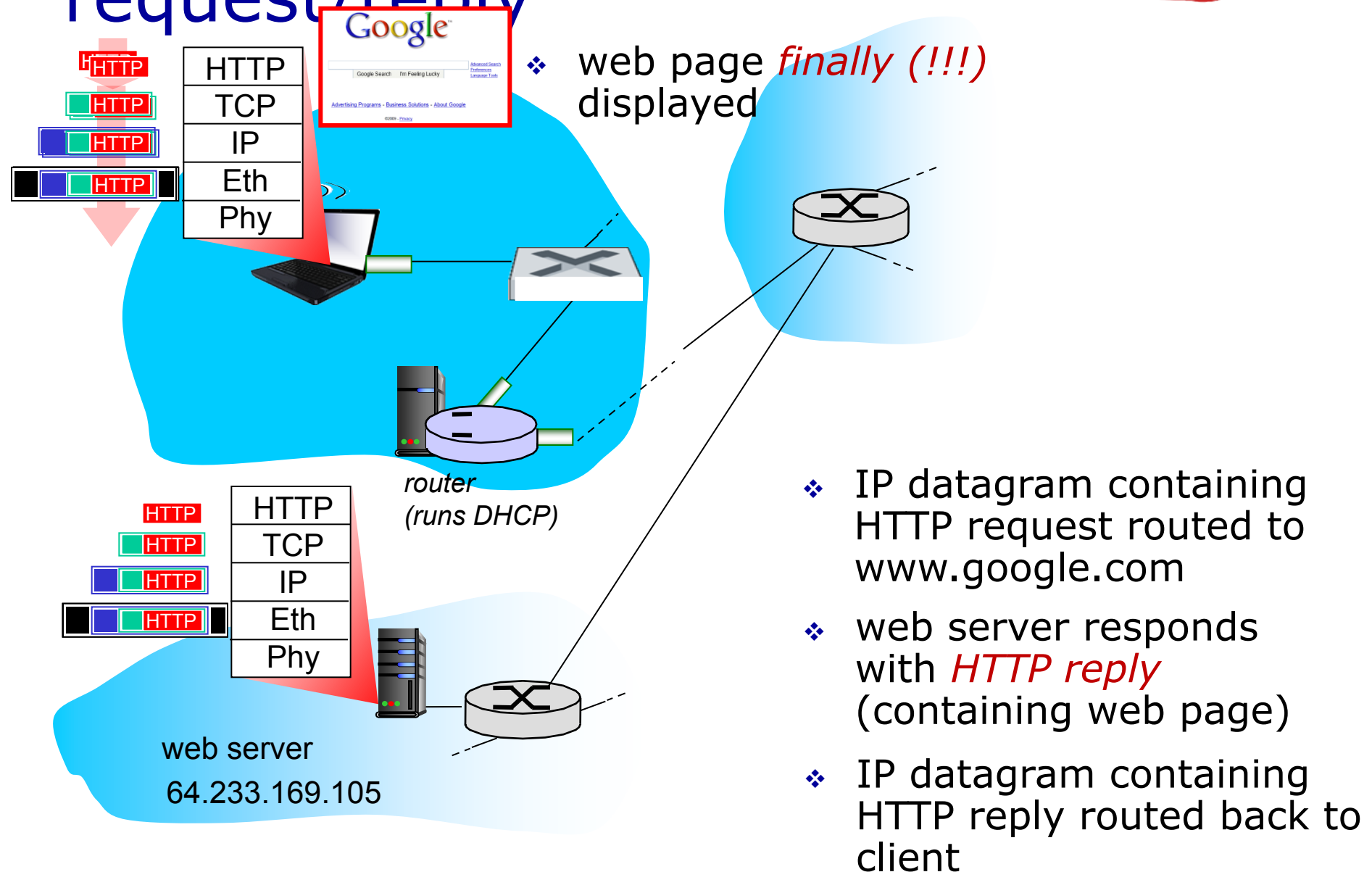
- ❖ IP datagram forwarded from campus network into comcast network, routed (tables created by *RIP*, *OSPF*, *IS-IS* and/or *BGP* routing protocols) to DNS server
- ❖ demuxed to DNS server
- ❖ DNS server replies to client with IP address of [www.google.com](http://www.google.com)

# A day in the life...TCP connection carrying ~~HTTP~~



# A day in the life... HTTP

## ~~request/reply~~



# Chapter 5: Summary

- ❖ principles behind data link layer services:
  - error detection, correction
  - sharing a bro multiple access
- ❖ insta various link layer technologies
  - Ethernet
  - switched LANS, VLANs
  - virtualized networks as a link layer: MPLS
- ❖ synthesis: a day in the life of a web request



# Chapter 5: let's take a ~~breath~~

- ❖ journey down protocol stack *complete* (except PHY)
- ❖ solid understanding of networking principles, pr

inter

- wireless
- multimedia
- security
- network management